

Minimum Spanning Tree

Gianluigi Zavattaro
Dip. di Informatica – Scienza e Ingegneria
Università di Bologna
gianluigi.zavattaro@unibo.it

Slide realizzate a partire da materiale fornito dal Prof. Moreno Marzolla

Original work Copyright © Alberto Montresor, Università di Trento, Italy
(<http://www.dit.unitn.it/~montreso/asd/index.shtml>)

Modifications Copyright © 2009—2011 Moreno Marzolla, Università di Bologna, Italy
(<http://www.moreno.marzolla.name/teaching/ASD2010/>)

This work is licensed under the Creative Commons Attribution-NonCommercial-ShareAlike License. To view a copy of this license, visit <http://creativecommons.org/licenses/by-nc-sa/2.5/> or send a letter to Creative Commons, 543 Howard Street, 5th Floor, San Francisco, California, 94105, USA.

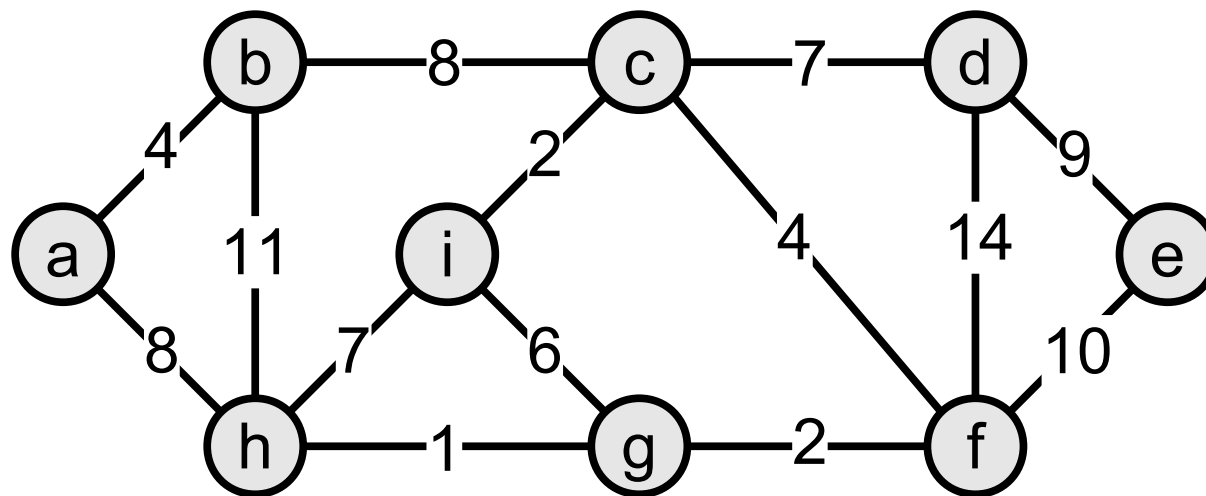
Introduzione

- Un problema di notevole importanza:
 - determinare come interconnettere diversi elementi fra loro minimizzando certi vincoli sulle connessioni
- Esempio classico:
 - progettazione dei circuiti elettronici dove si vuole minimizzare la quantità di filo elettrico per collegare fra loro i diversi componenti
- Questo problema prende il nome di:
 - albero di copertura (di peso) minimo
 - albero di connessione (di peso) minimo
 - **minimum spanning tree**

Definizione del problema

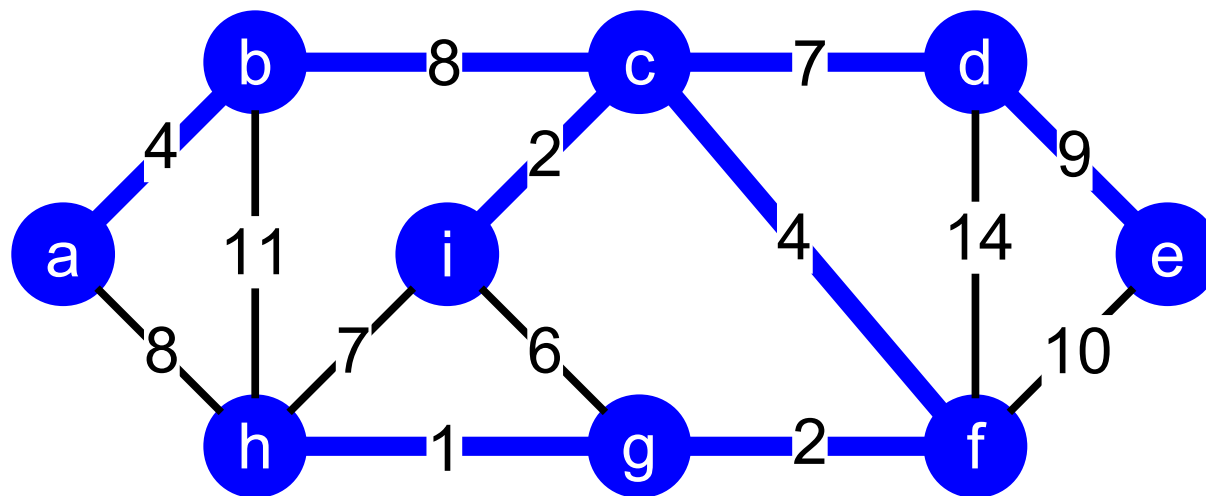
- Input:

- $G = (V, E)$ un grafo non orientato e connesso
- $w: V \times V \rightarrow R$ una funzione peso
 - se $\{u, v\} \in E$, allora $w(u, v)$ è il peso dell'arco $\{u, v\}$
 - se $\{u, v\} \notin E$, allora $w(u, v) = \infty$
- Poiché G non è orientato, $w(u, v) = w(v, u)$



Definizione del problema

- Albero di copertura (spanning tree)
 - Dato un grafo $G = (V, E)$ non orientato e connesso, un *albero di copertura* di G è un sottografo $T = (V, E_T)$ tale che
 - T è un albero
 - $E_T \subseteq E$
 - T contiene tutti i nodi di G



Definizione del problema

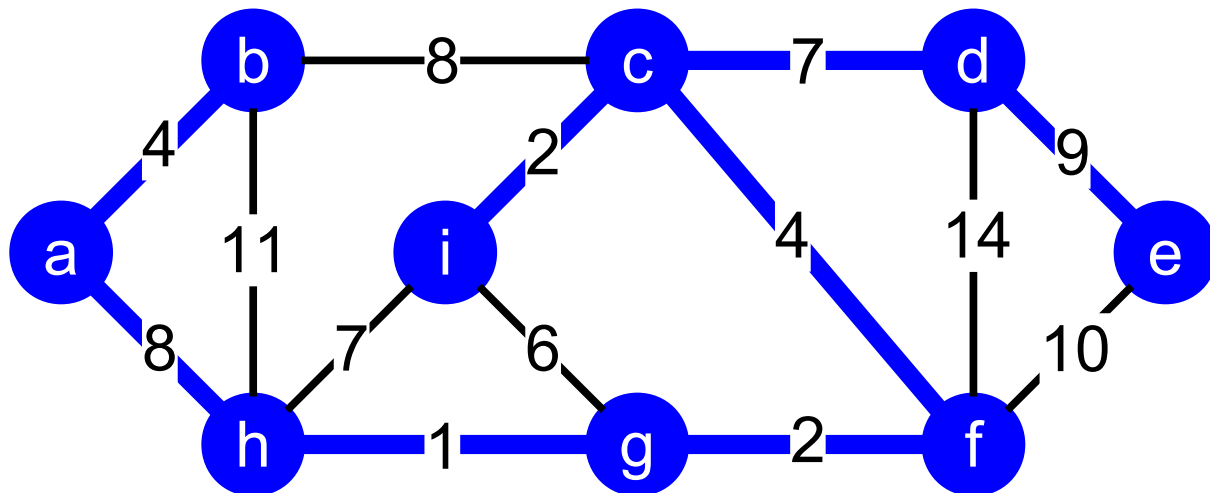
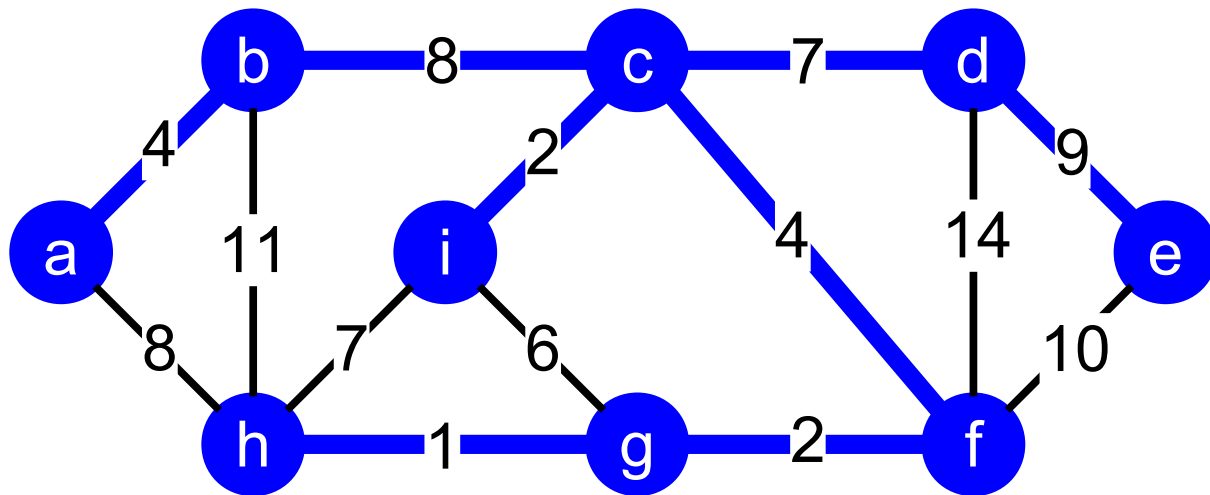
- Output: albero di copertura di peso minimo (minimum spanning tree)
 - Un albero di copertura T il cui peso totale

$$w(T) = \sum_{(u,v) \in T} w(u,v)$$

sia minimo, tra tutti i possibili alberi di copertura

Osservazione

- Il MST non è necessariamente unico



Metodo generico per calcolare MST

- Vediamo
 - Un metodo greedy **generico**
 - Due algoritmi che seguono questo metodo: **Kruskal** e **Prim**
- L'idea è di **accrescere** un sottoinsieme T di archi in modo tale che venga rispettata la seguente condizione:
 - T è un sottoinsieme di qualche albero di copertura minimo
- Un arco $\{u, v\}$ è detto **sicuro** per T se $T \cup \{u, v\}$ è ancora un sottoinsieme di qualche MST

Metodo generico per calcolare MST

```
Tree Generic-MST(Grafo  $G=(V,E,w)$ )  
  Tree  $T \leftarrow$  Albero vuoto  
  while  $T$  non forma un albero di copertura do  
    trova un arco sicuro  $\{u, v\}$   
     $T \leftarrow T \cup \{u, v\}$   
  endwhile  
  return  $T$ 
```

- Archi **blu**
 - sono gli archi che fanno parte del MST
- Archi **rossi**
 - sono gli archi che non fanno parte del MST

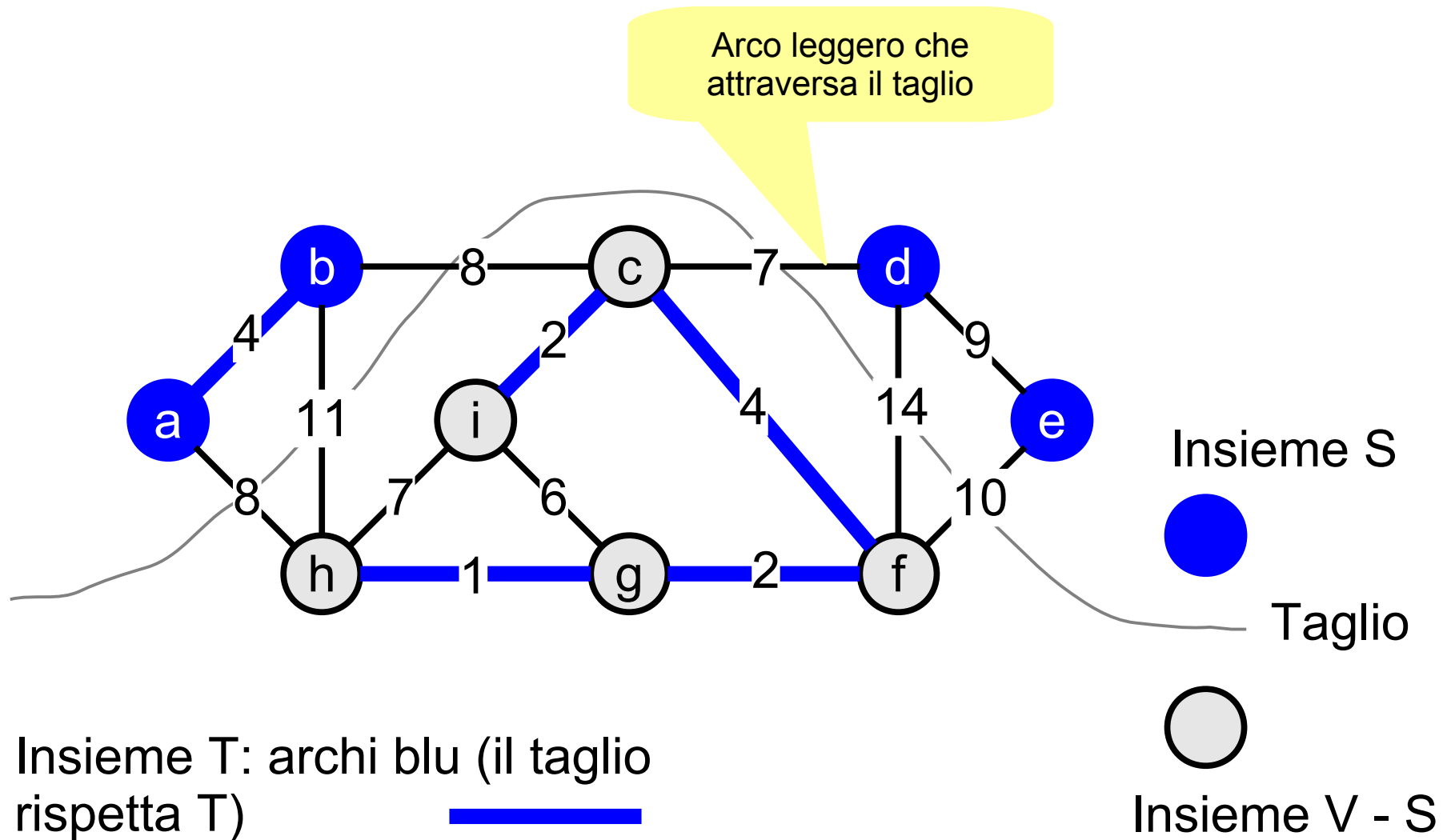
Definizioni

- Per caratterizzare gli archi sicuri dobbiamo introdurre alcune definizioni:
 - Un **taglio** $(S, V - S)$ di un grafo non orientato $G = (V, E)$ è una partizione di V in due sottoinsiemi disgiunti
 - Un arco $\{u, v\}$ **attraversa il taglio** se $u \in S$ e $v \in V - S$
 - Un taglio **rispetta** un insieme di archi T se nessun arco di T attraversa il taglio
 - Un arco che attraversa un taglio è **leggero** se il suo peso è minimo fra i pesi degli archi che attraversano un taglio

Regole del ciclo e del taglio

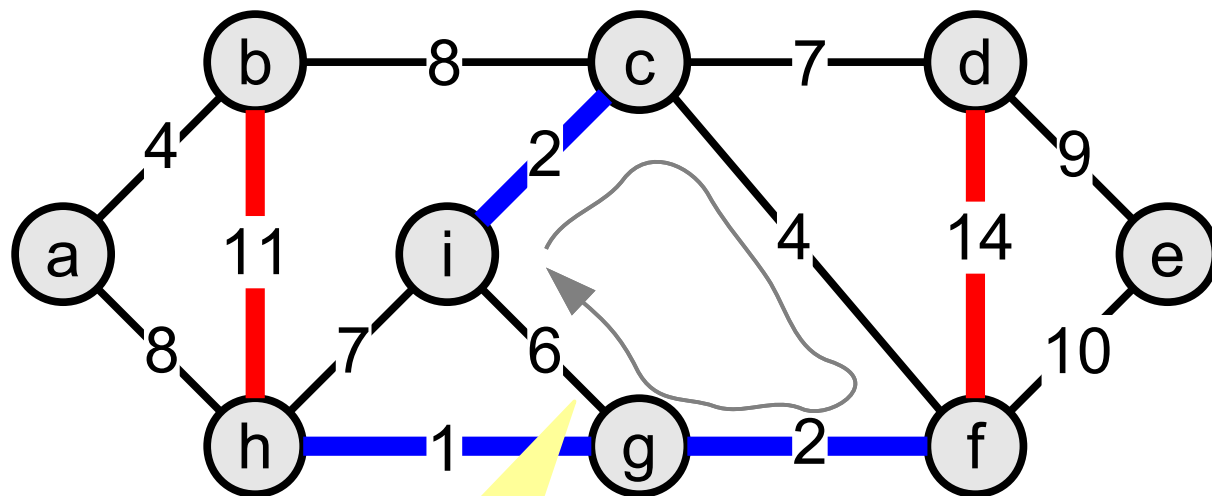
- Regola del **taglio**
 - Scegli un taglio in G che **rispetta** gli archi già colorati di blu (non attraversato da archi blu). Tra tutti gli archi non colorati che attraversano il taglio selezionane uno **leggero** (di peso minimo) e coloralo di blu
- Regola del **ciclo**
 - Scegli un ciclo semplice in G che **non contenga archi rossi**. Tra tutti gli archi non colorati del ciclo, seleziona un arco di costo massimo e coloralo di rosso
- Si può costruire un MST usando tali regole:
 - Costruisce un MST applicando in successione una delle due regole precedenti (una qualunque, purché si possa usare)

Applicazione regola del taglio



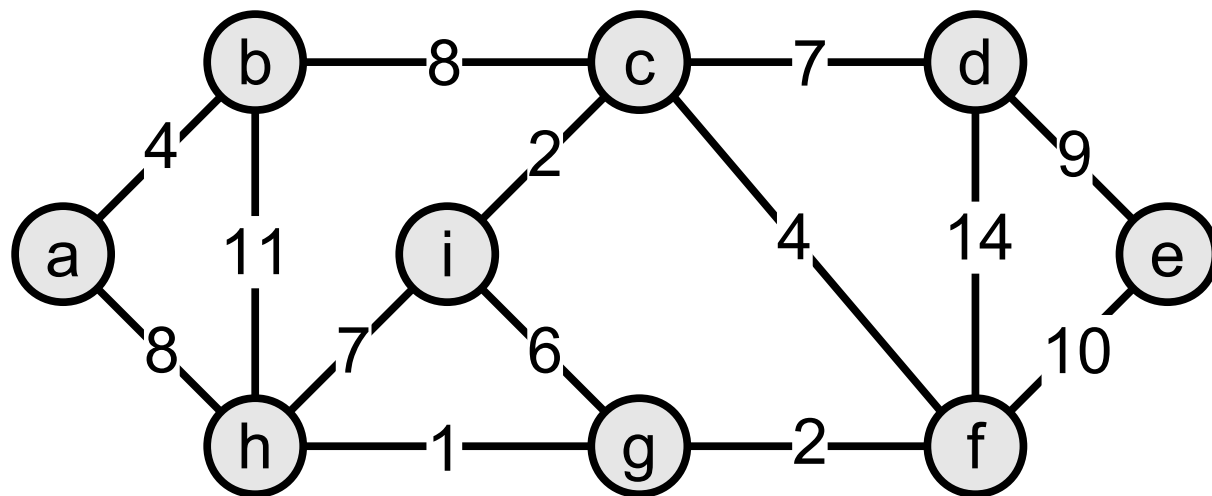
Applicazione regola del ciclo

Seleziona un ciclo semplice
che non contiene archi **rossi**

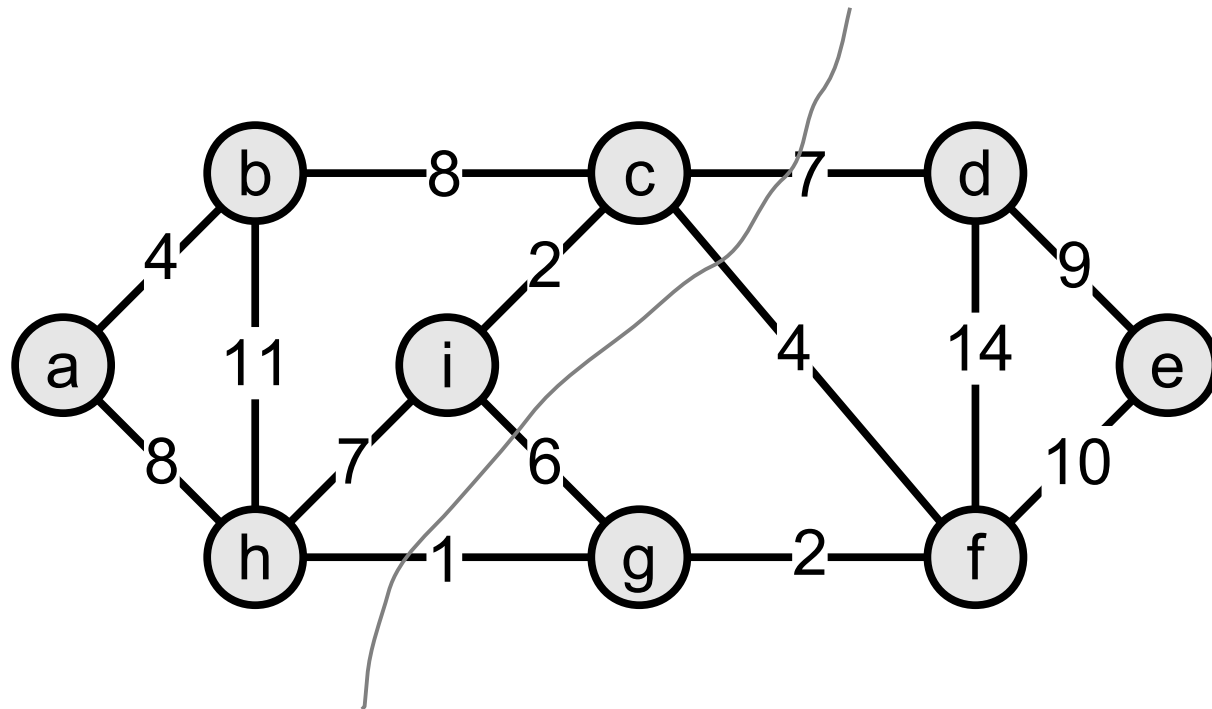


L'arco di peso massimo
del ciclo può essere
colorato di rosso

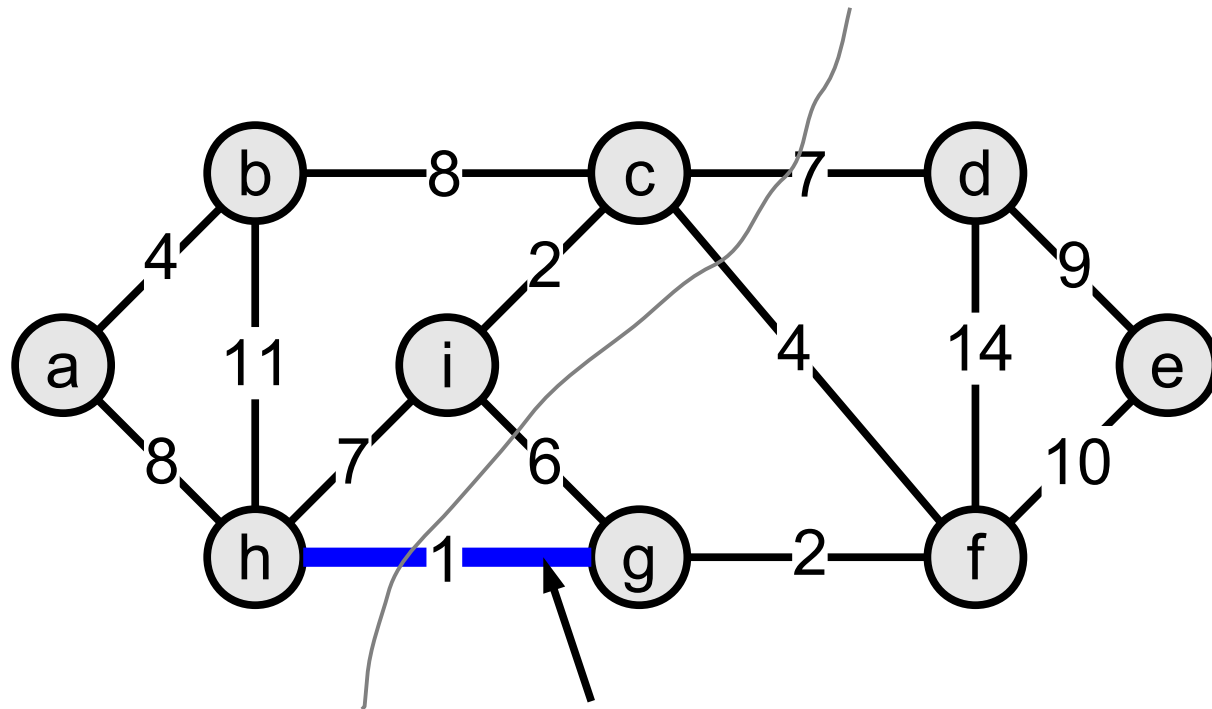
Esempio



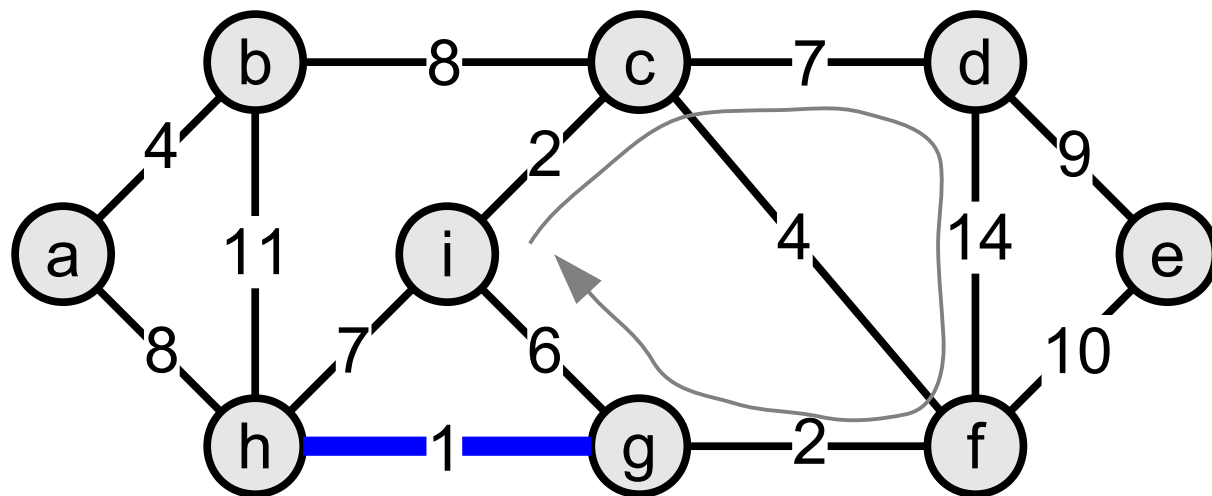
Esempio



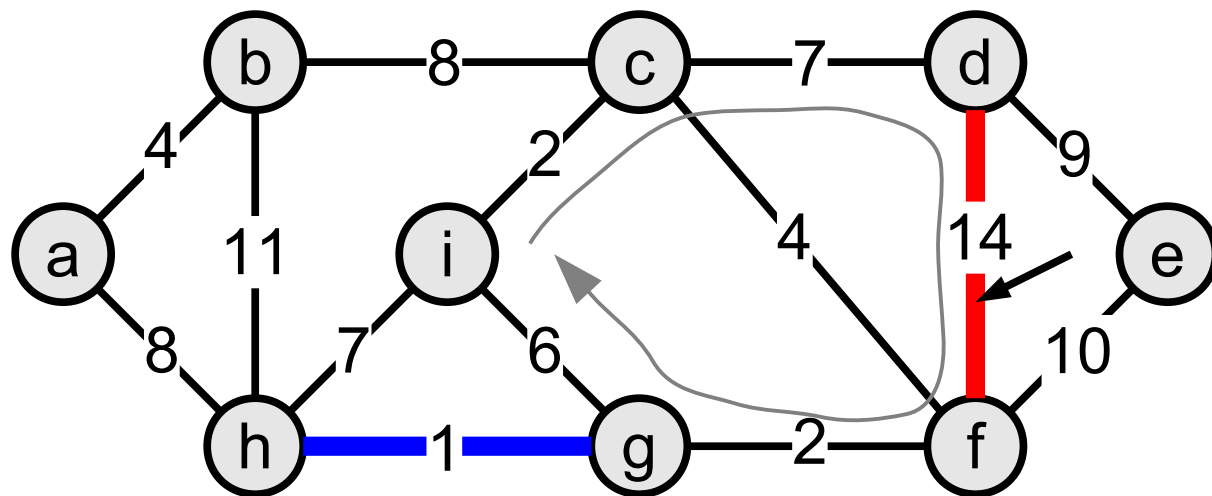
Esempio



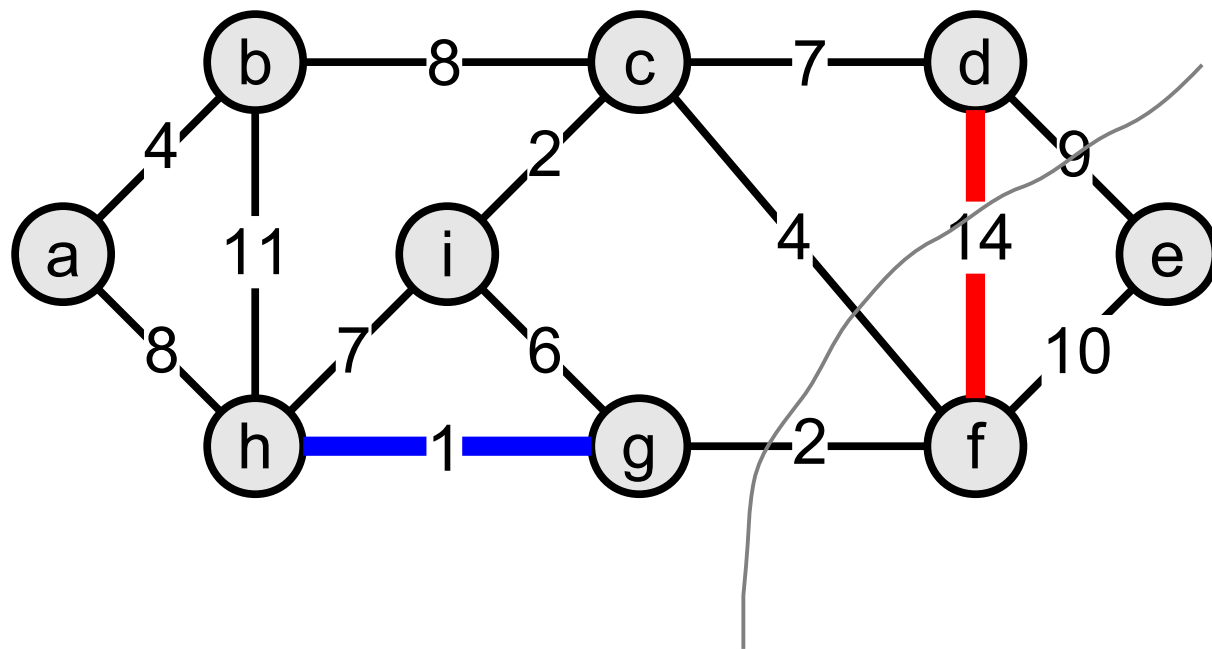
Esempio



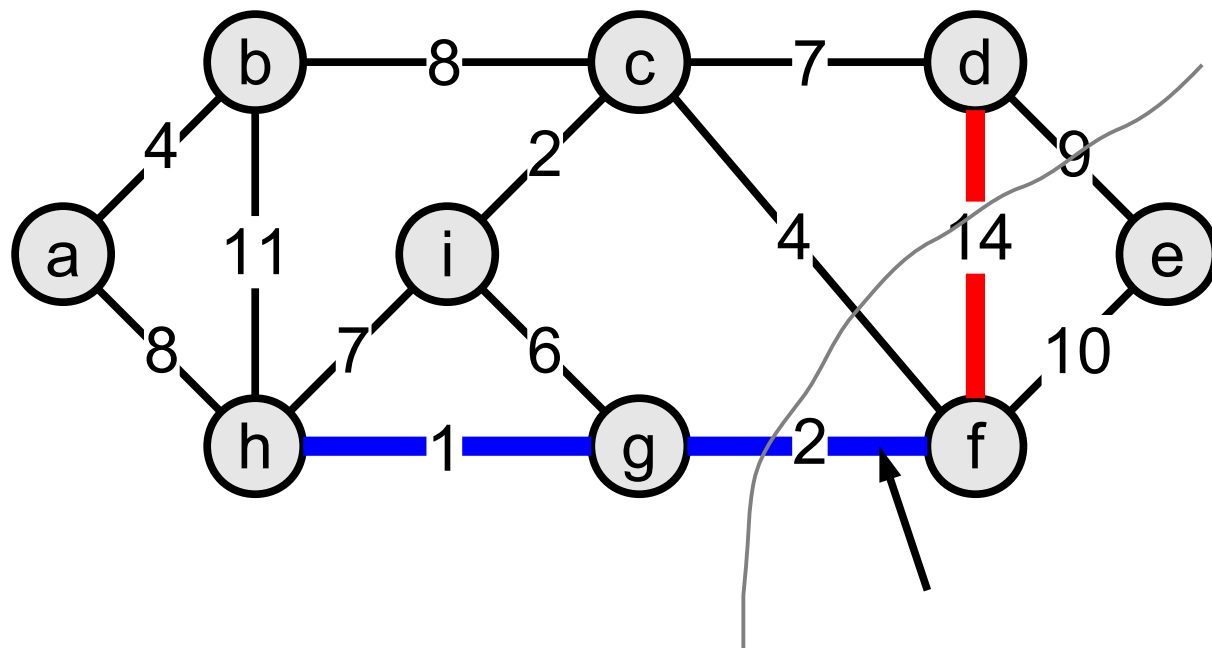
Esempio



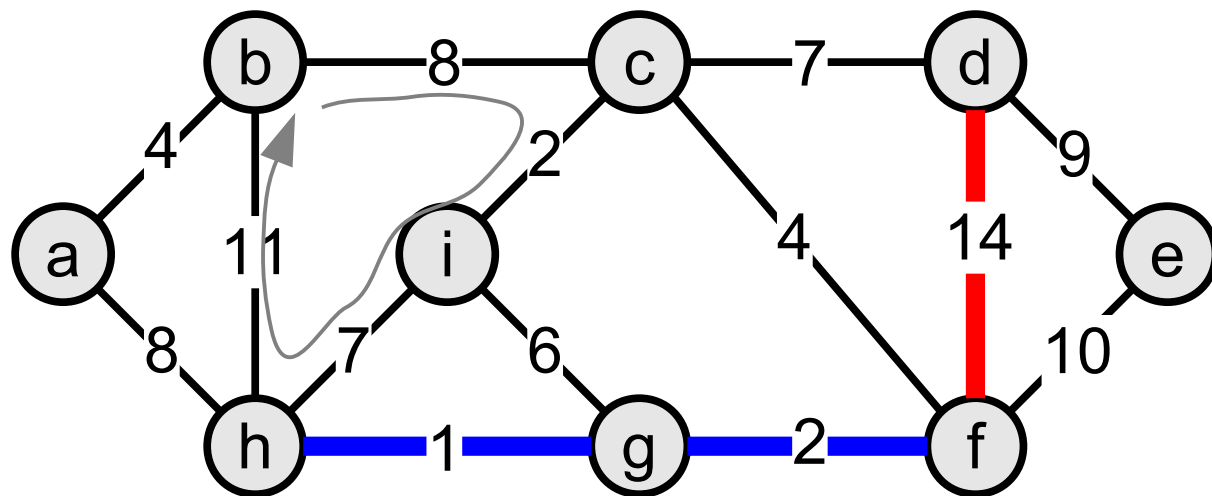
Esempio



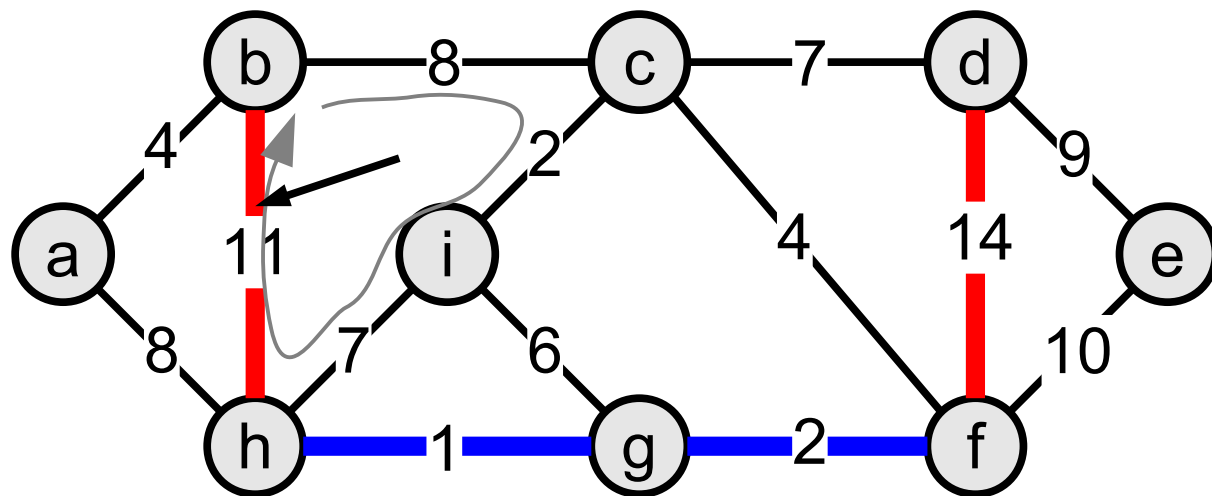
Esempio



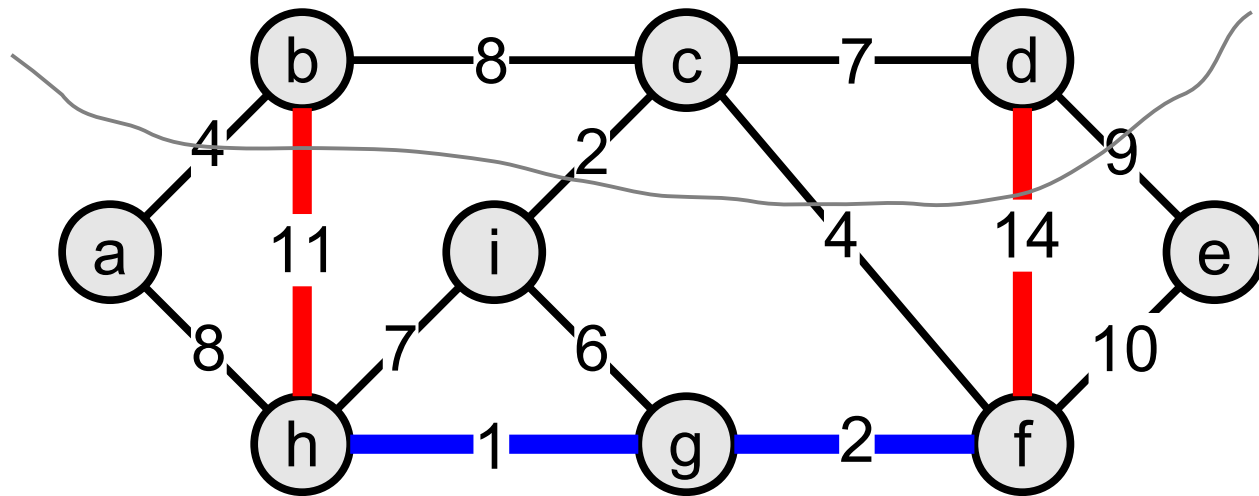
Esempio



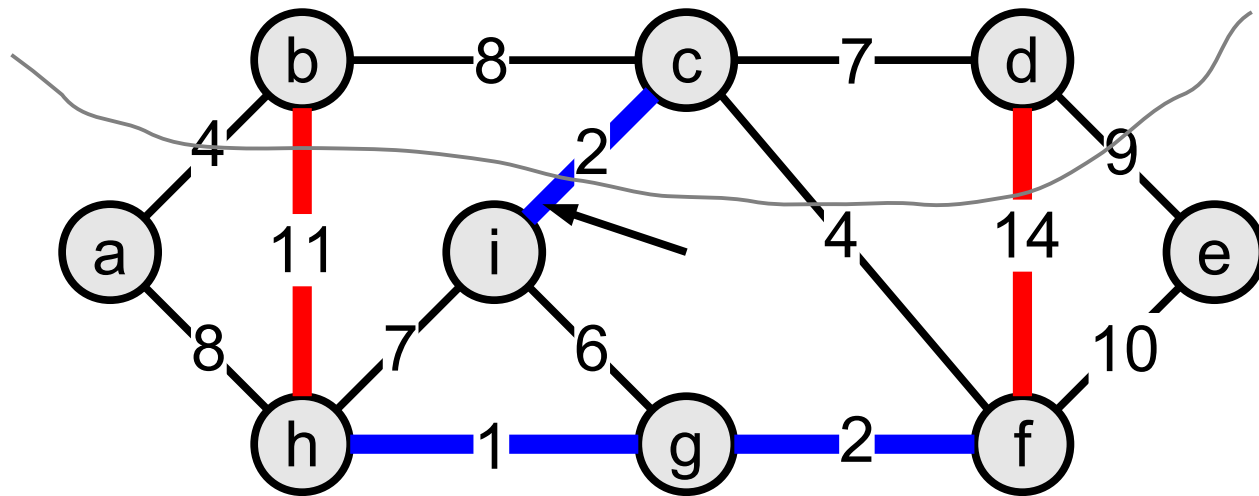
Esempio



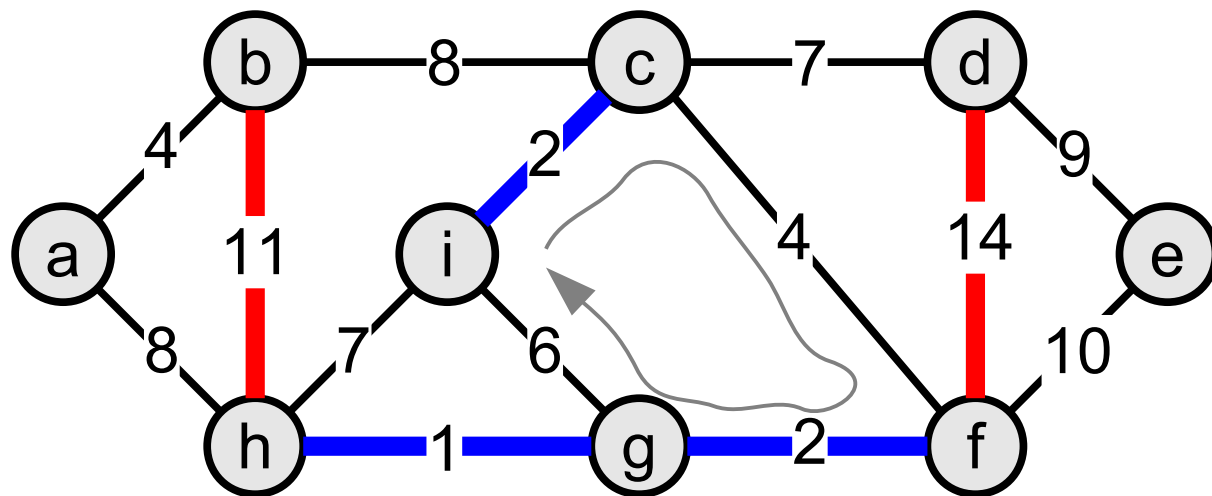
Esempio



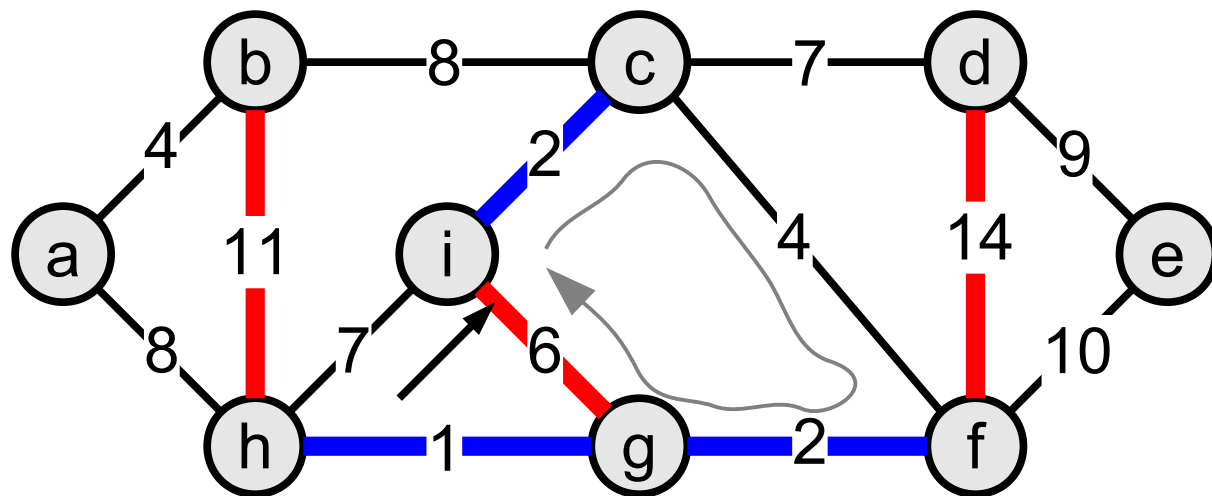
Esempio



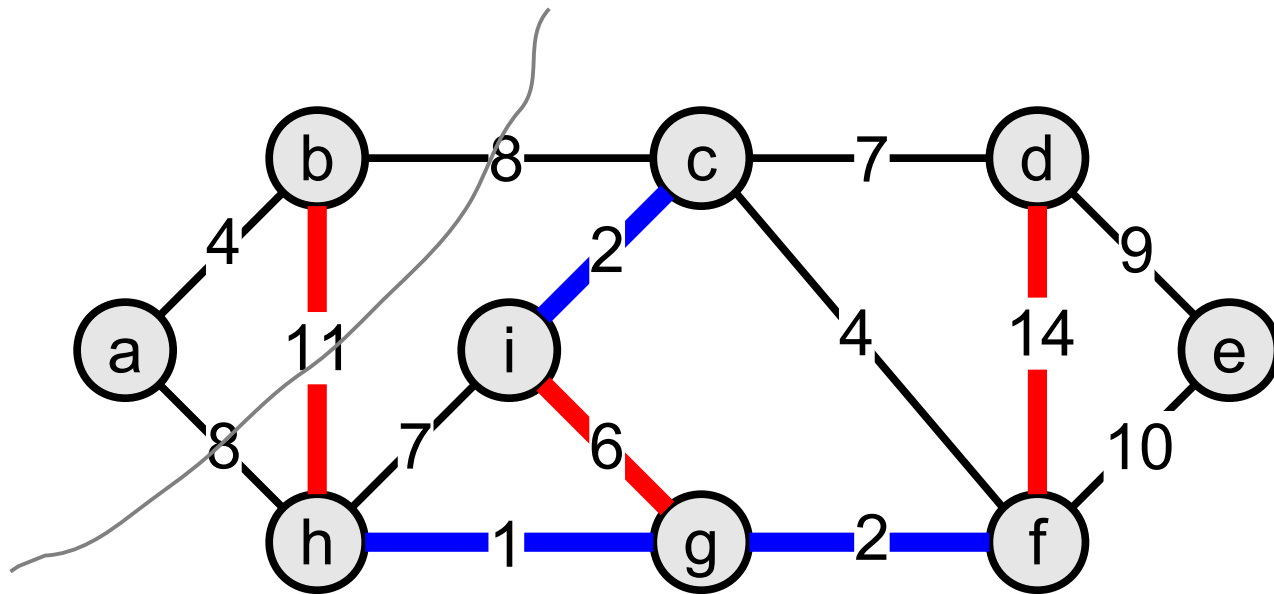
Esempio



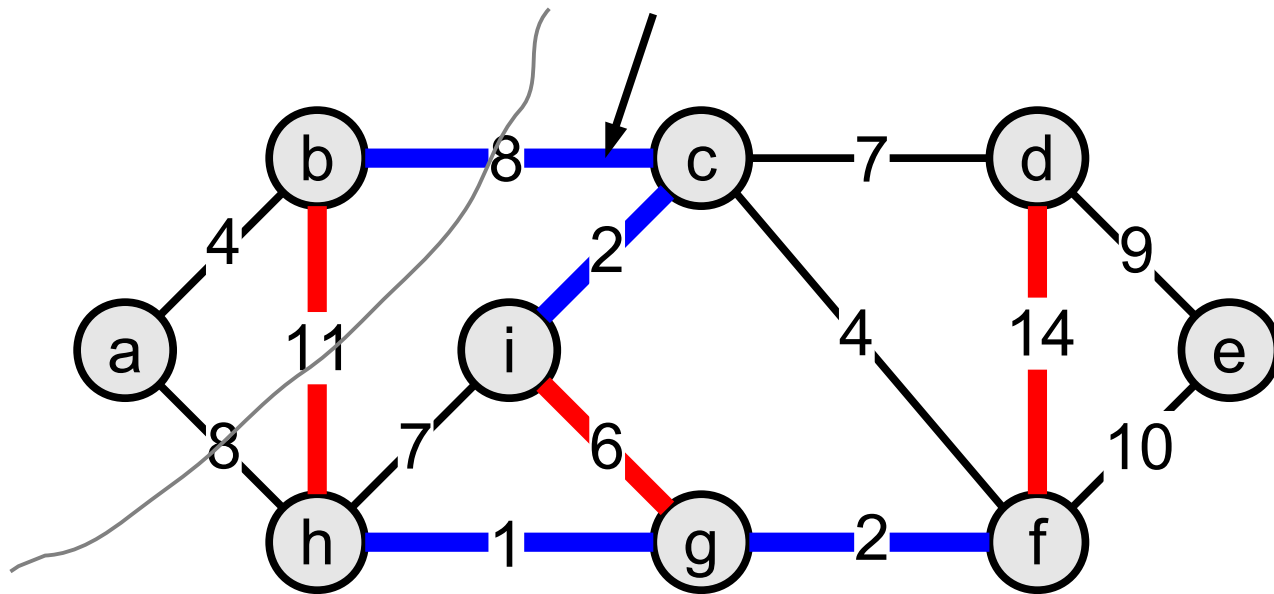
Esempio



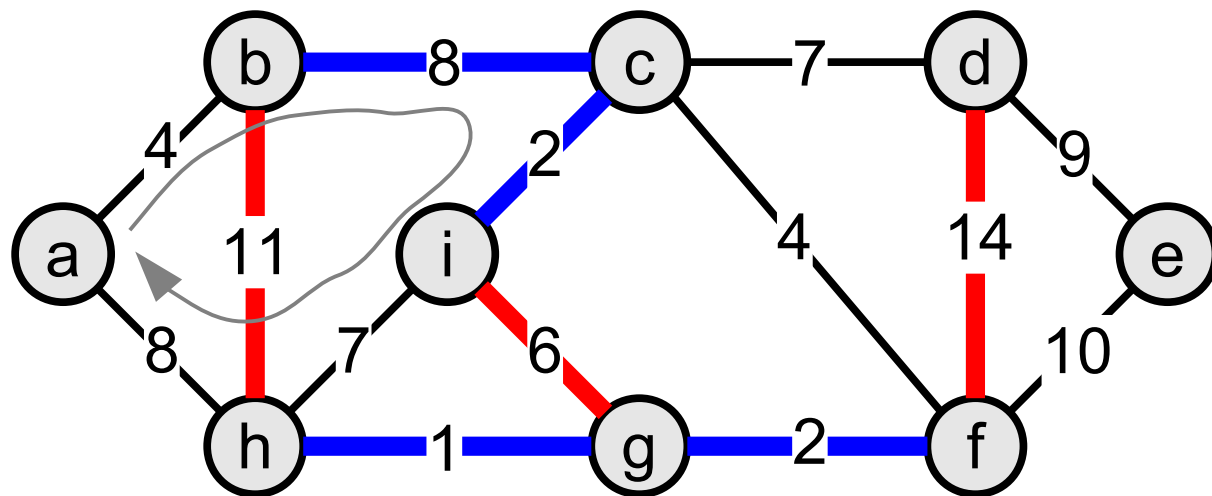
Esempio



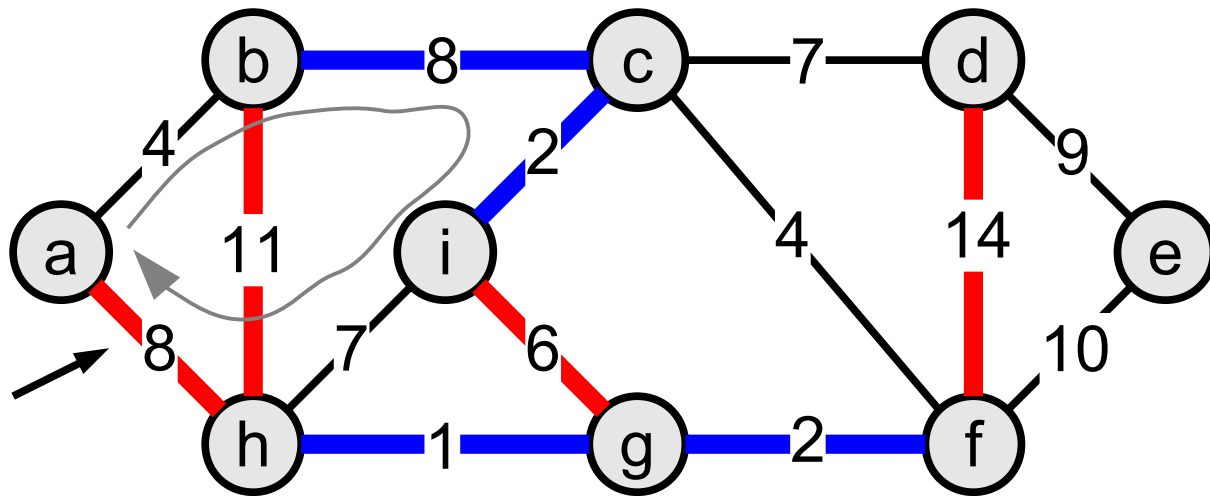
Esempio



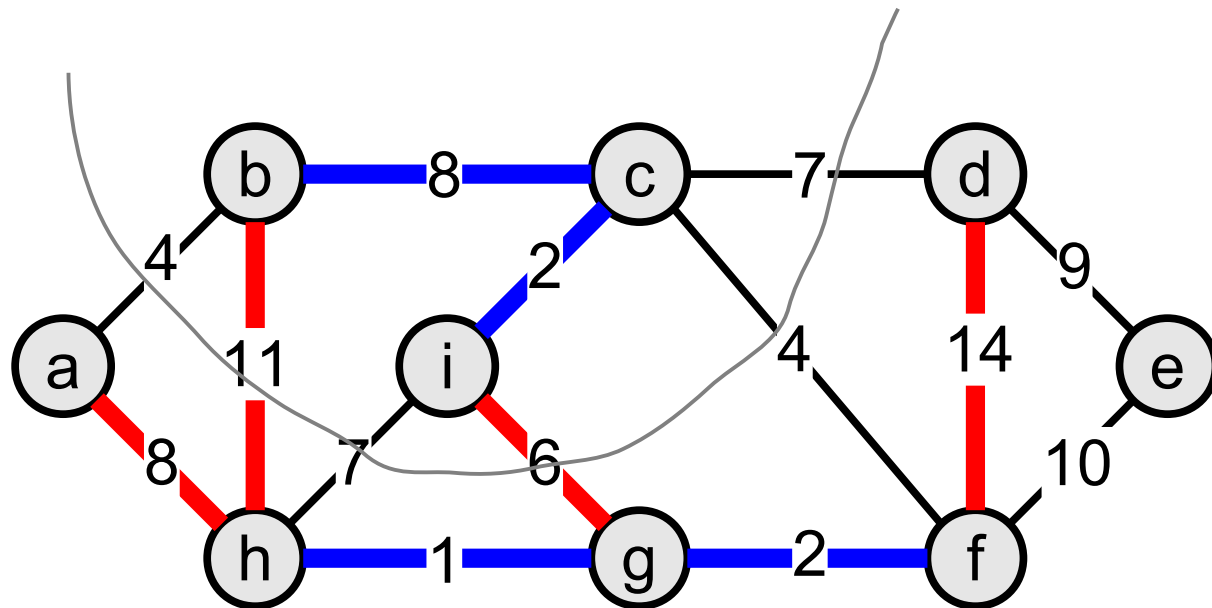
Esempio



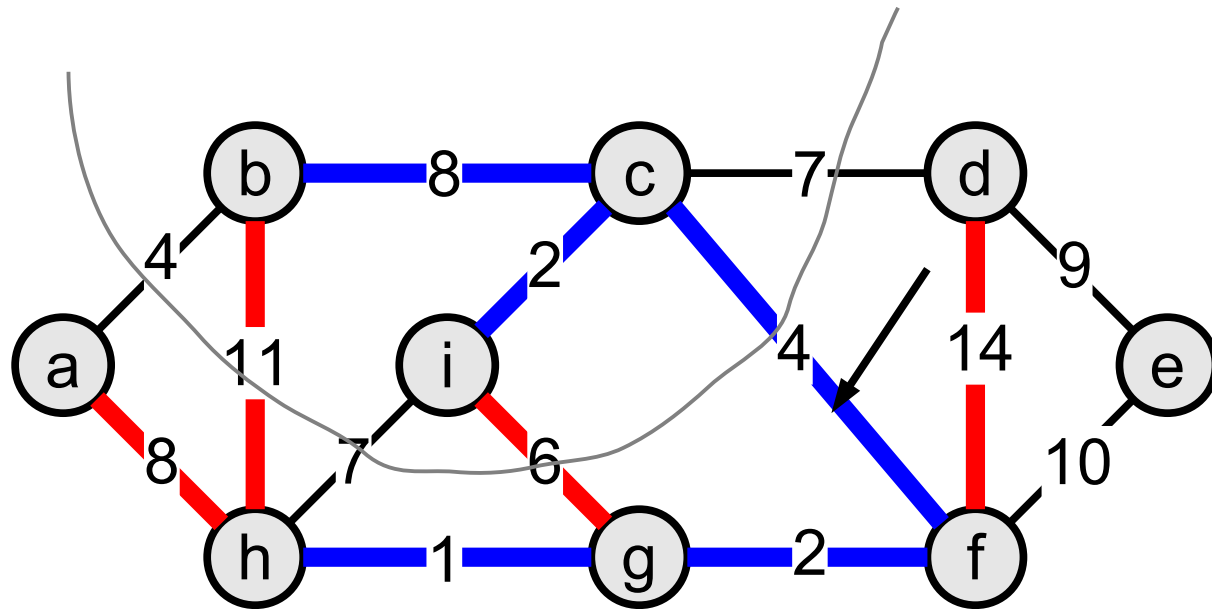
Esempio



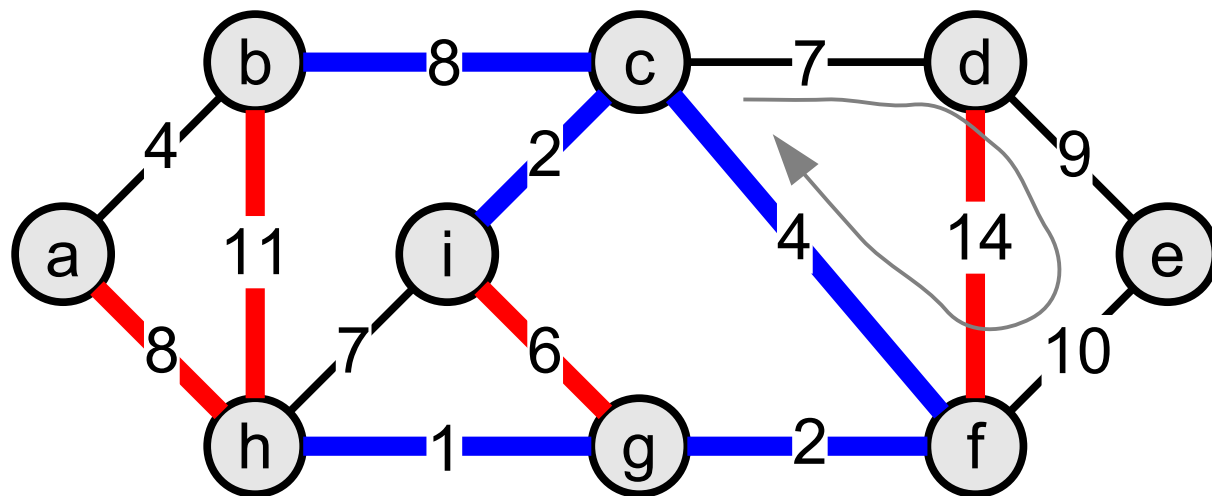
Esempio



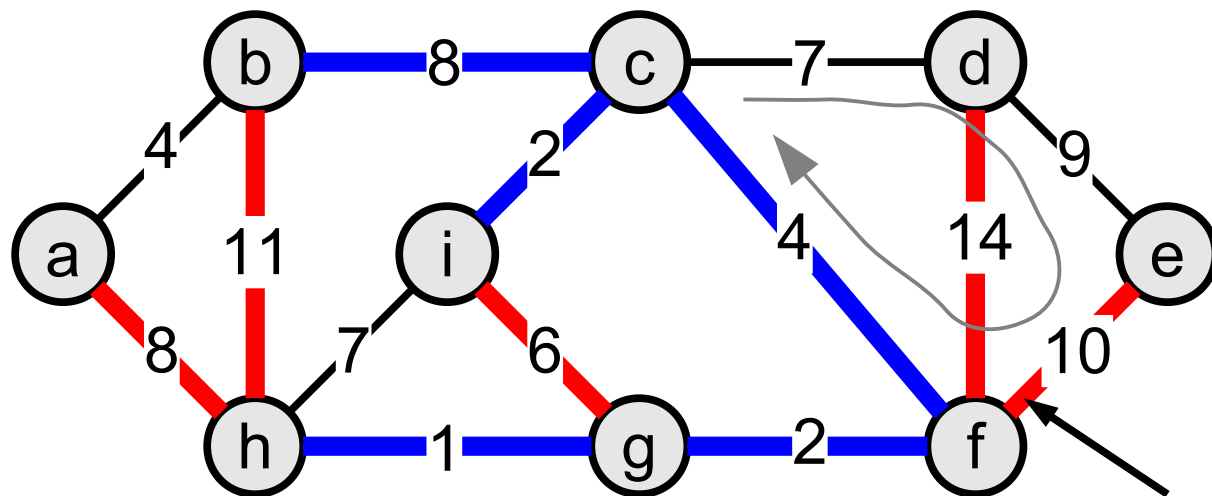
Esempio



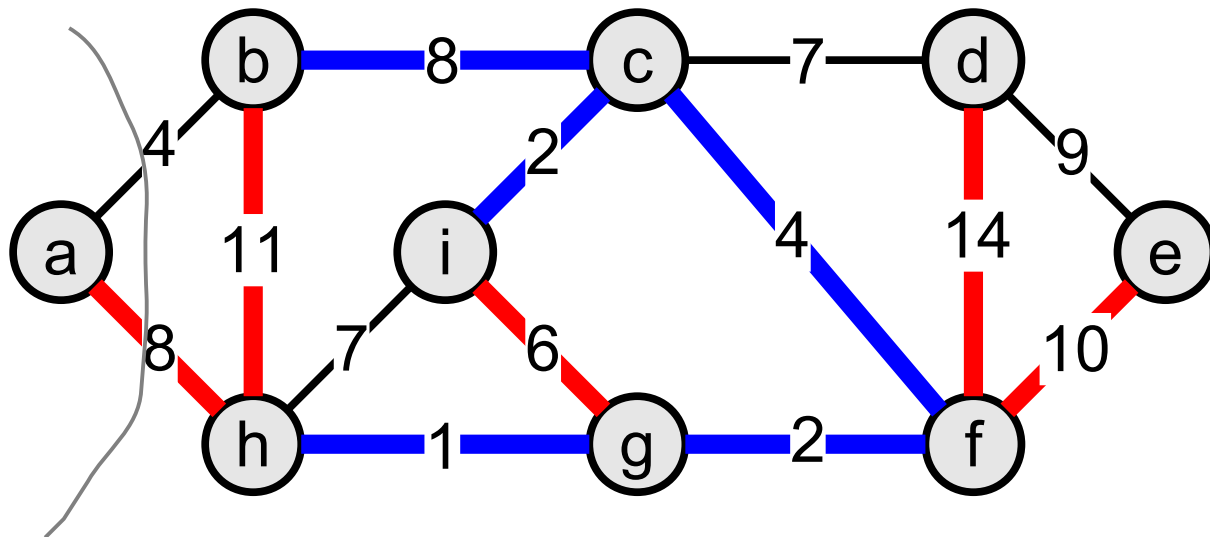
Esempio



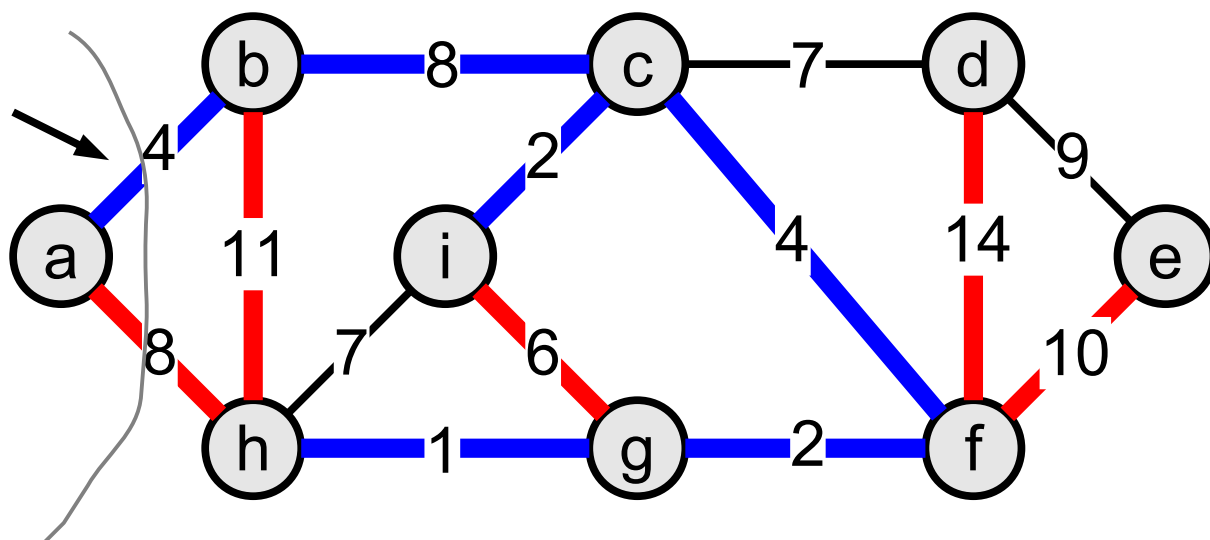
Esempio



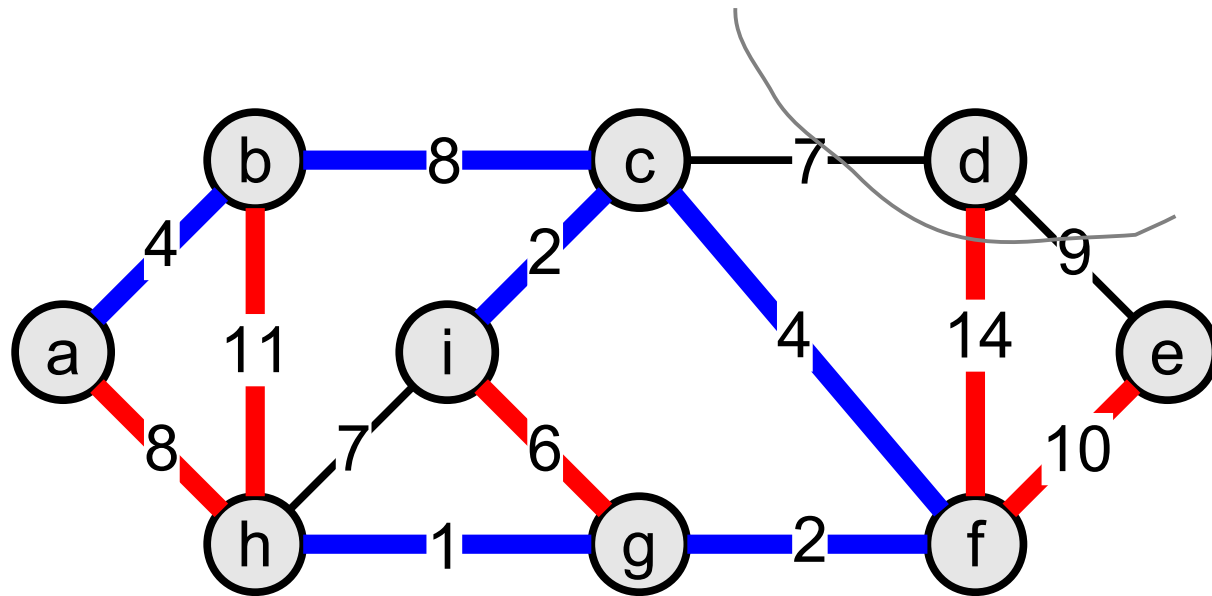
Esempio



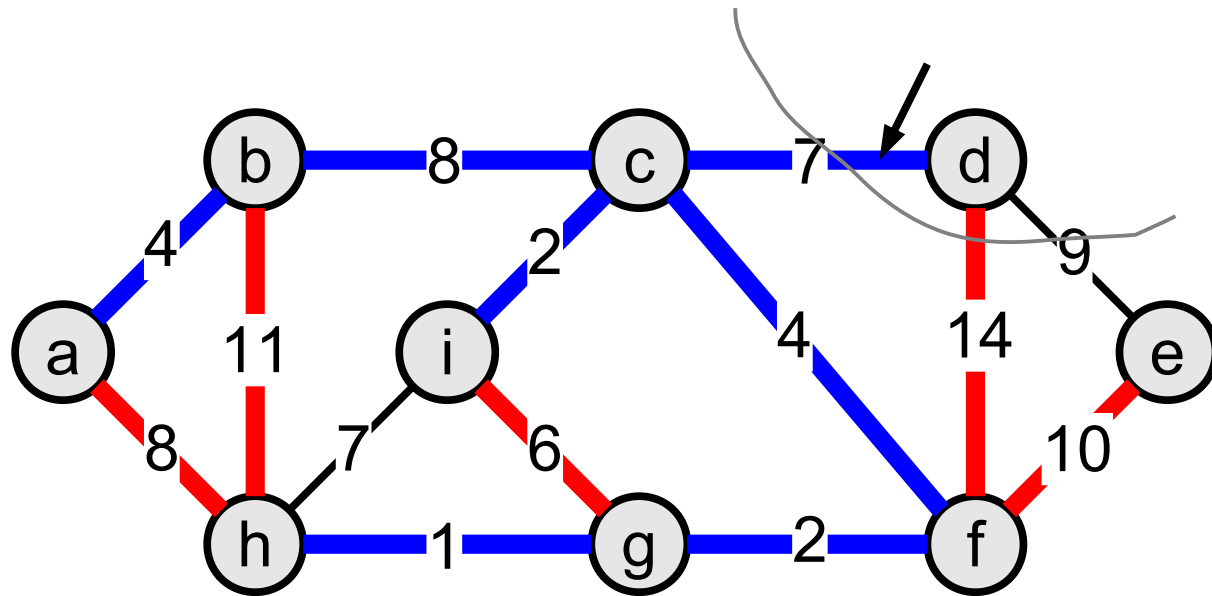
Esempio



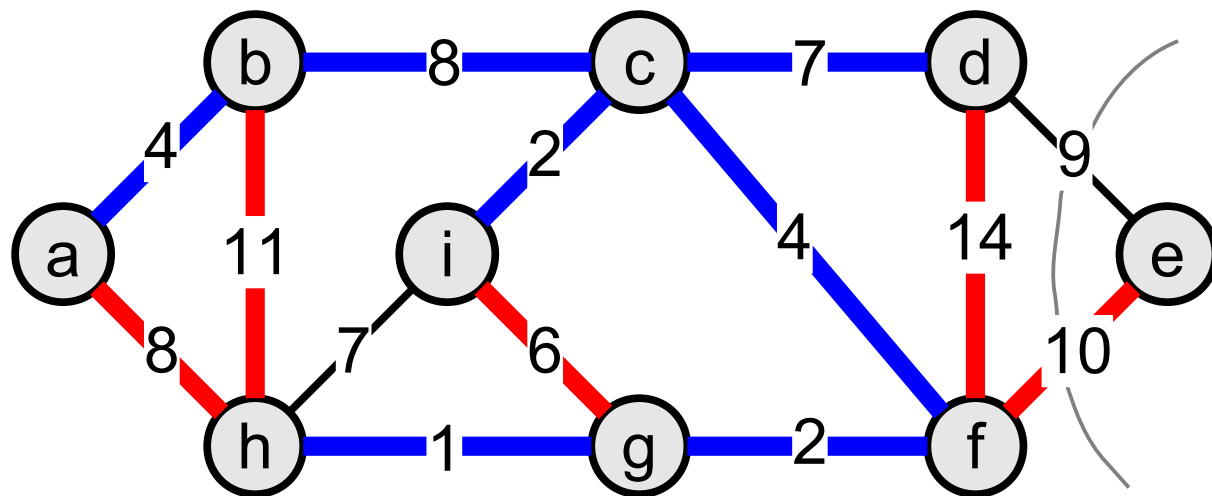
Esempio



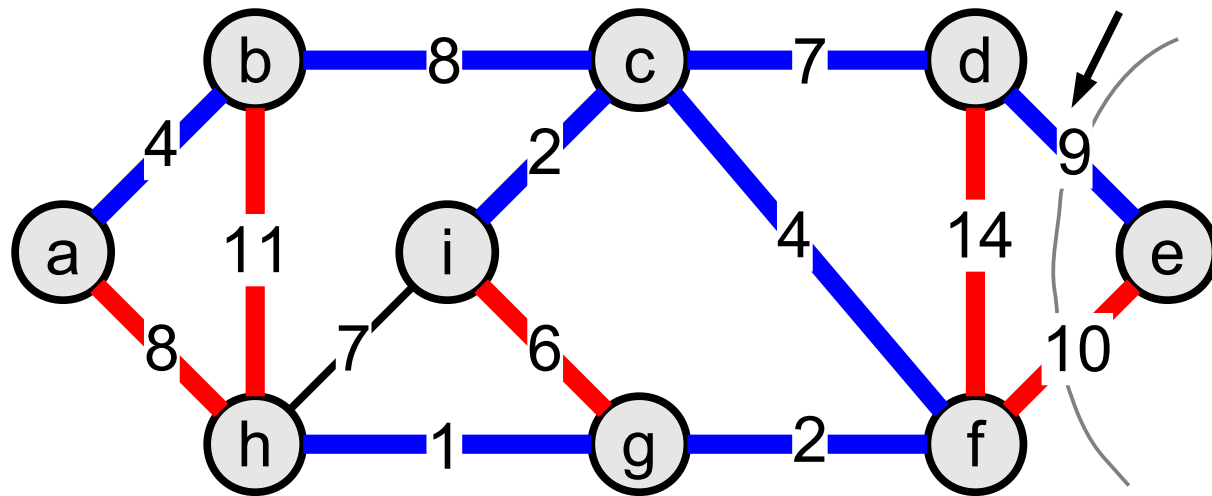
Esempio



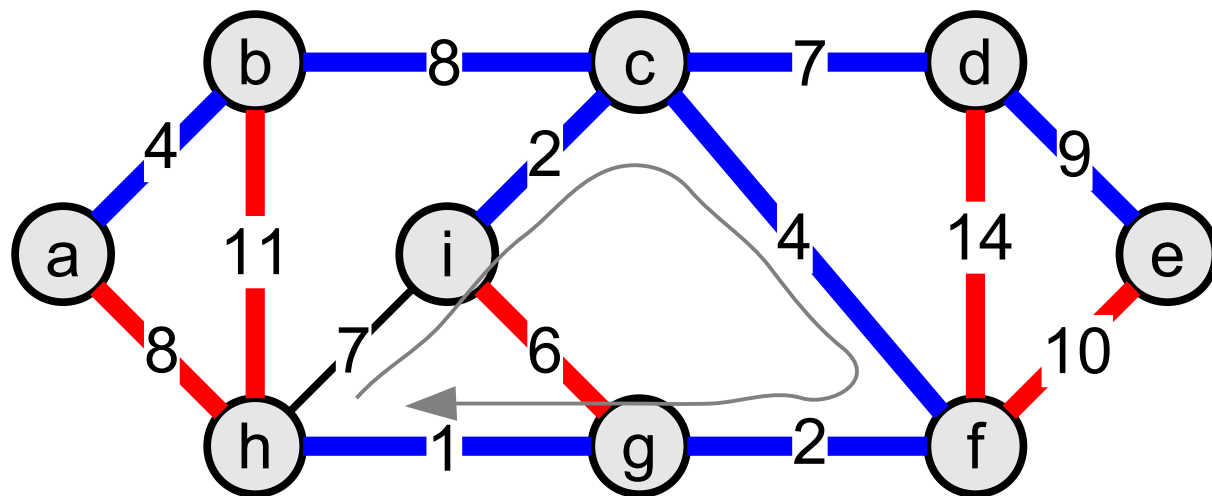
Esempio



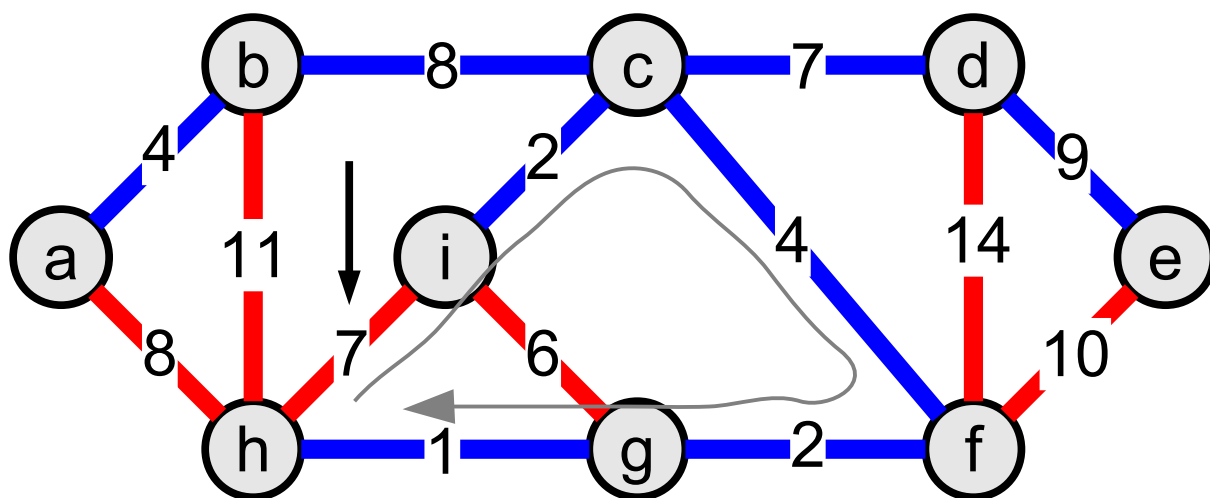
Esempio



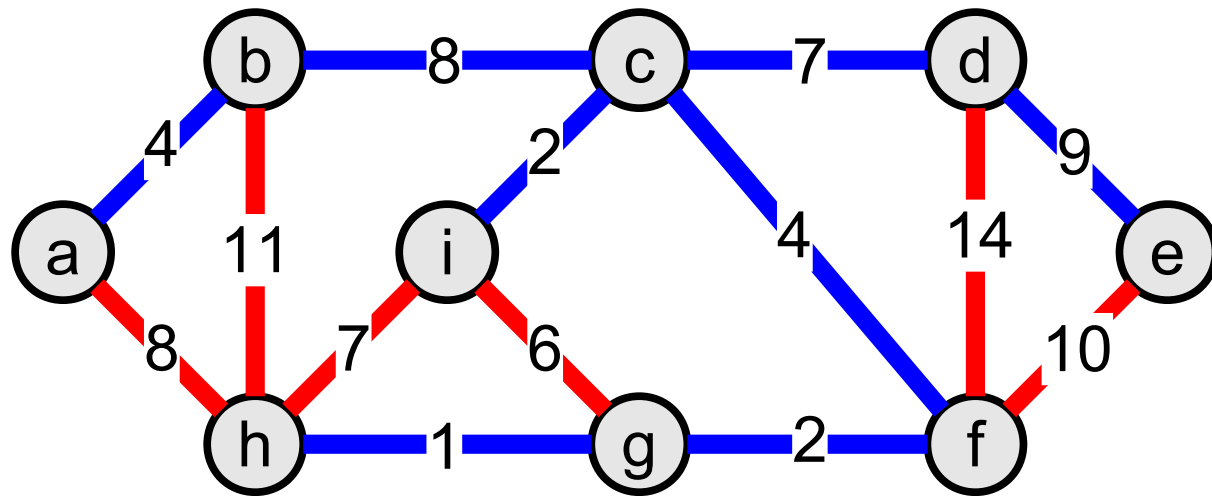
Esempio



Esempio



Finito!



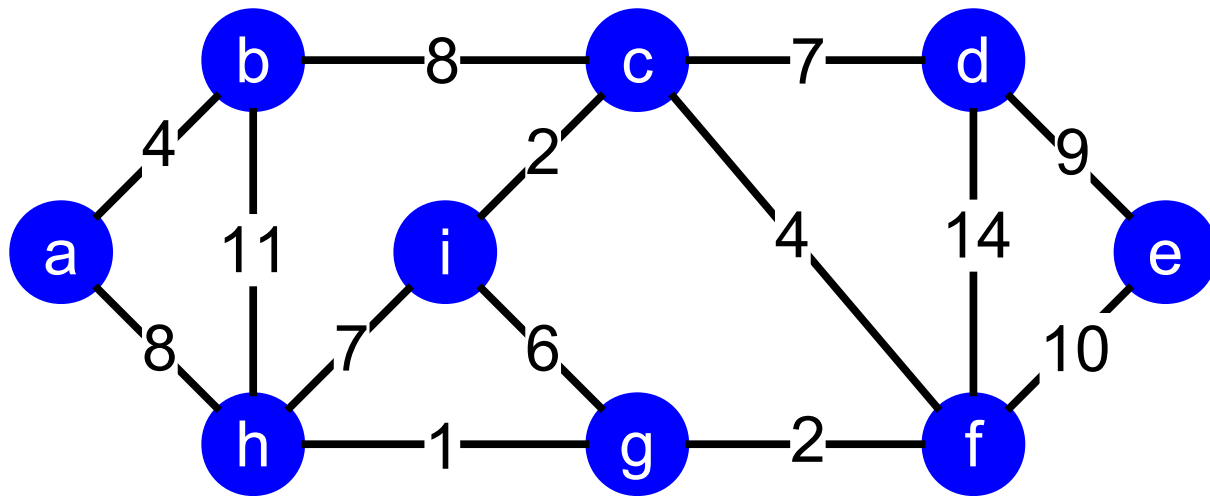
Algoritmo di Kruskal

- Quanto visto in precedenza non è un algoritmo (non indica in modo deterministico come applicare le regole)
- Kruskal fissa un ordine di applicazione delle regole:
 - Idea: ingrandire sottoinsiemi disgiunti di un albero di copertura minimo connettendoli fra di loro fino ad avere l'albero finale
 - Inizialmente la **foresta di copertura** è composta da n alberi, uno per ciascun nodo, e nessun arco
 - Si considerano gli archi in ordine non decrescente di peso
 - Se l'arco $e = \{u, v\}$ connette due alberi blu distinti, lo si colora di blu. Altrimenti lo si colora di rosso
 - L'algoritmo è greedy perché ad ogni passo si aggiunge alla foresta un arco con il peso minimo

Joseph B. Kruskal: *On the Shortest Spanning Subtree of a Graph and the Traveling Salesman Problem*. In: Proceedings of the American Mathematical Society, Vol 7, No. 1 (Feb, 1956), pp. 48–50

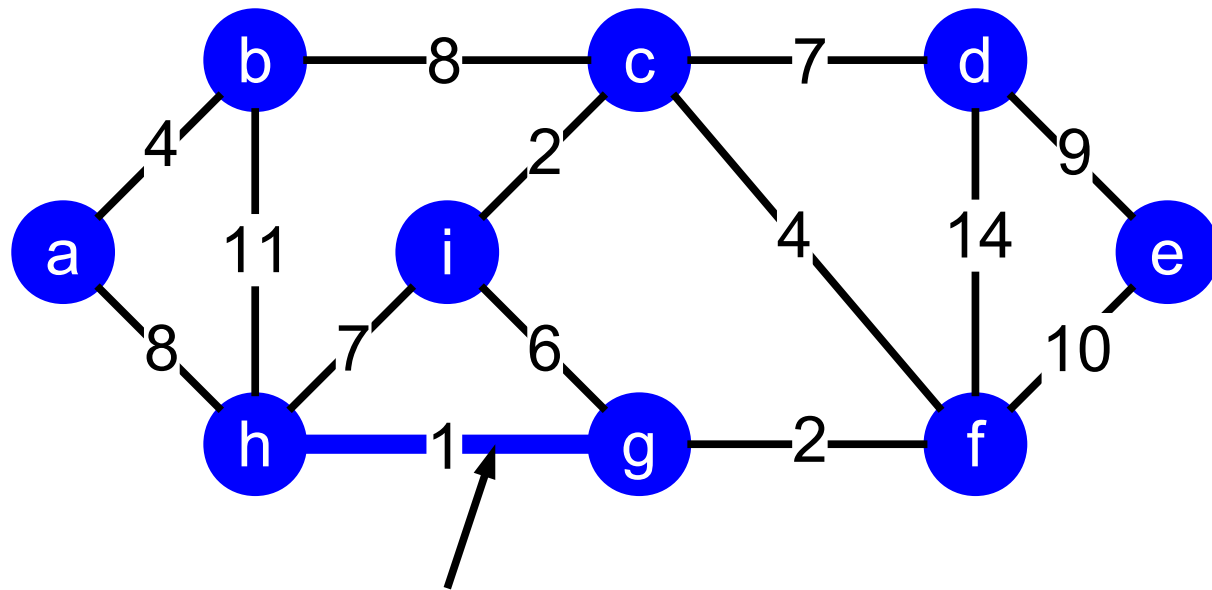
Esempio

Algoritmo di Kruskal



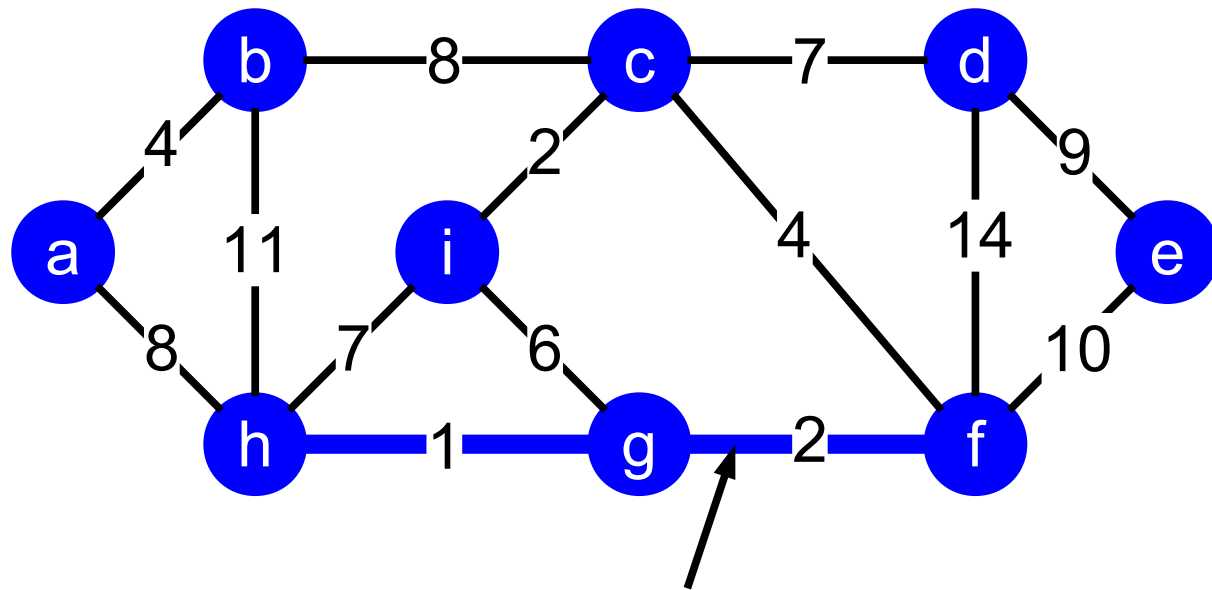
Esempio

Algoritmo di Kruskal



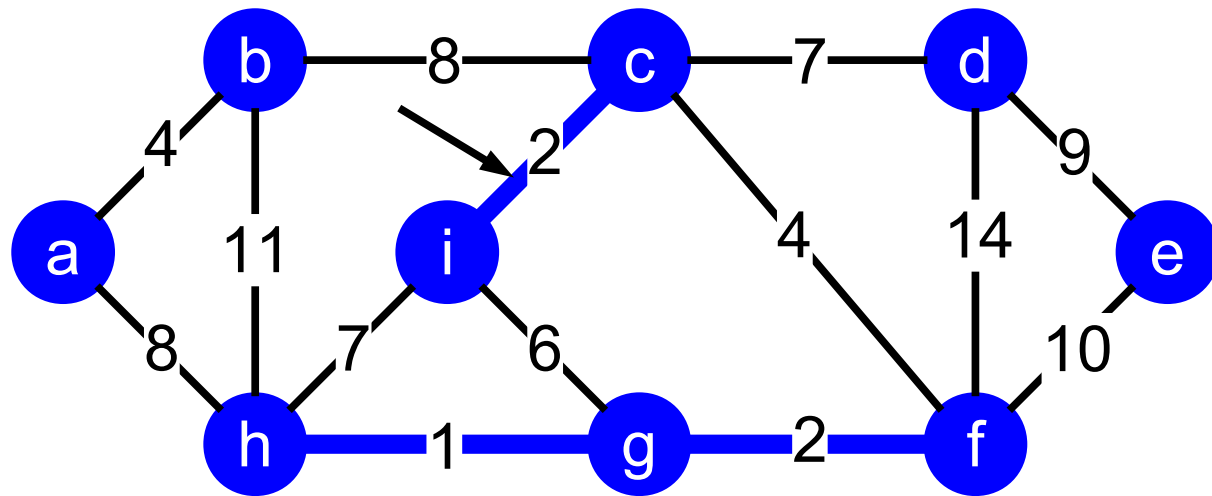
Esempio

Algoritmo di Kruskal



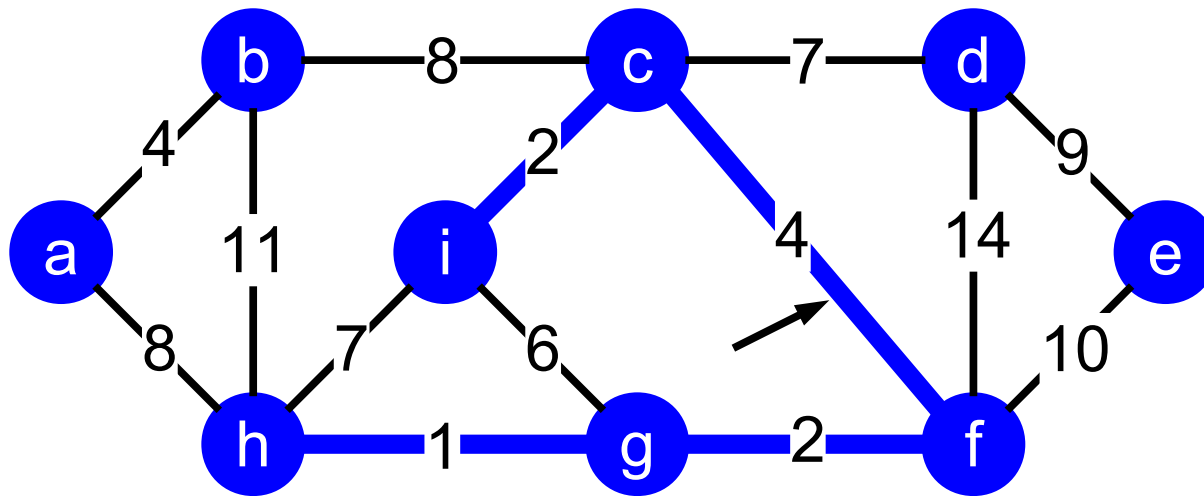
Esempio

Algoritmo di Kruskal



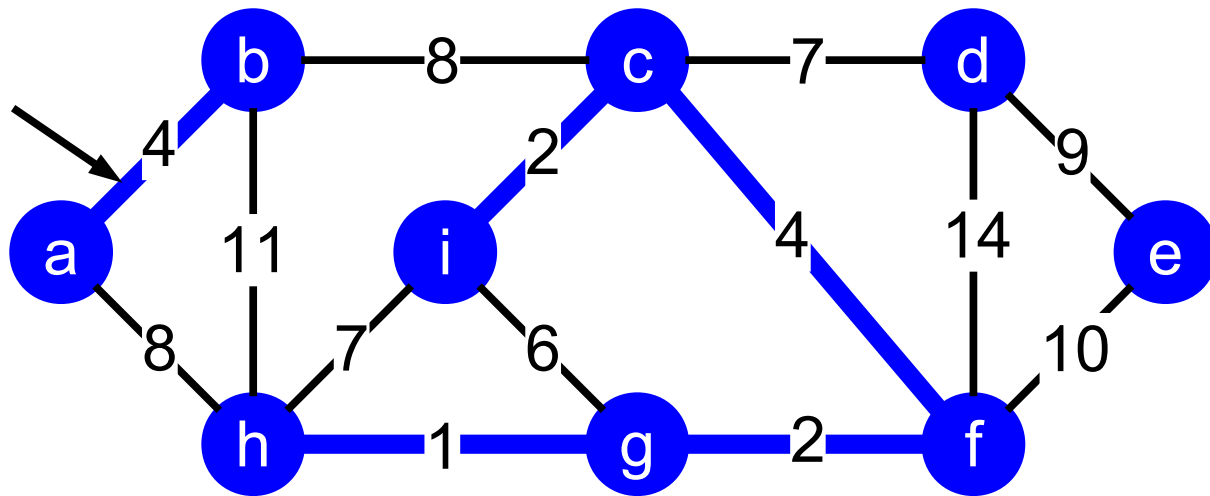
Esempio

Algoritmo di Kruskal



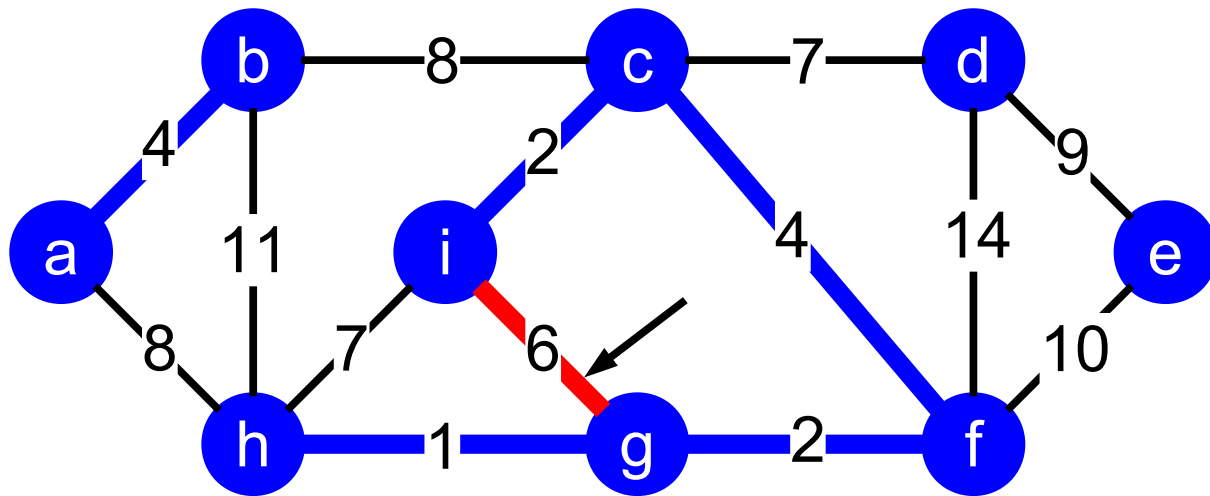
Esempio

Algoritmo di Kruskal



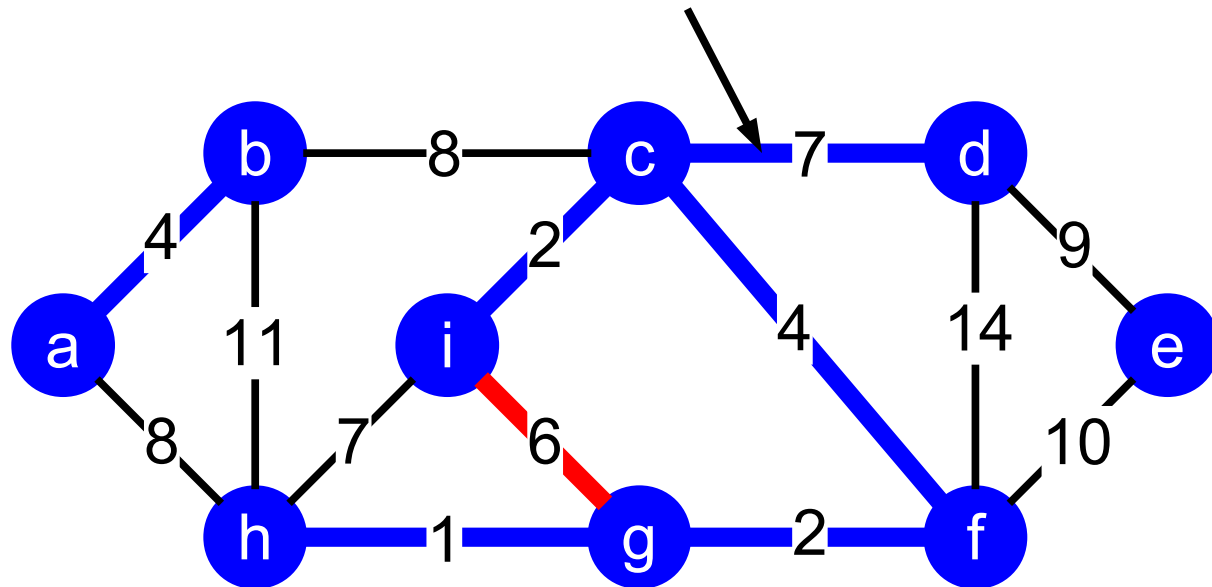
Esempio

Algoritmo di Kruskal



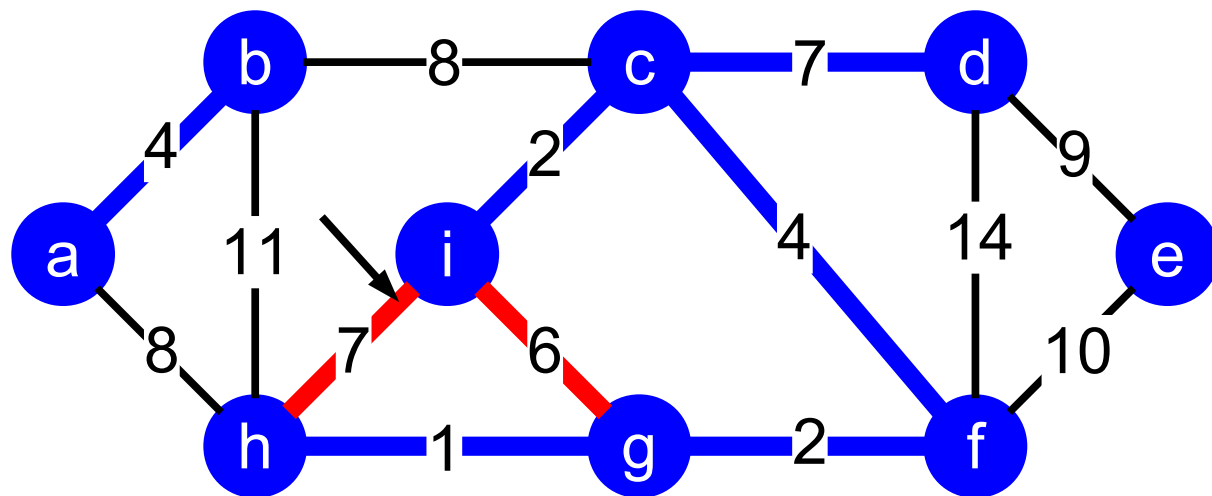
Esempio

Algoritmo di Kruskal



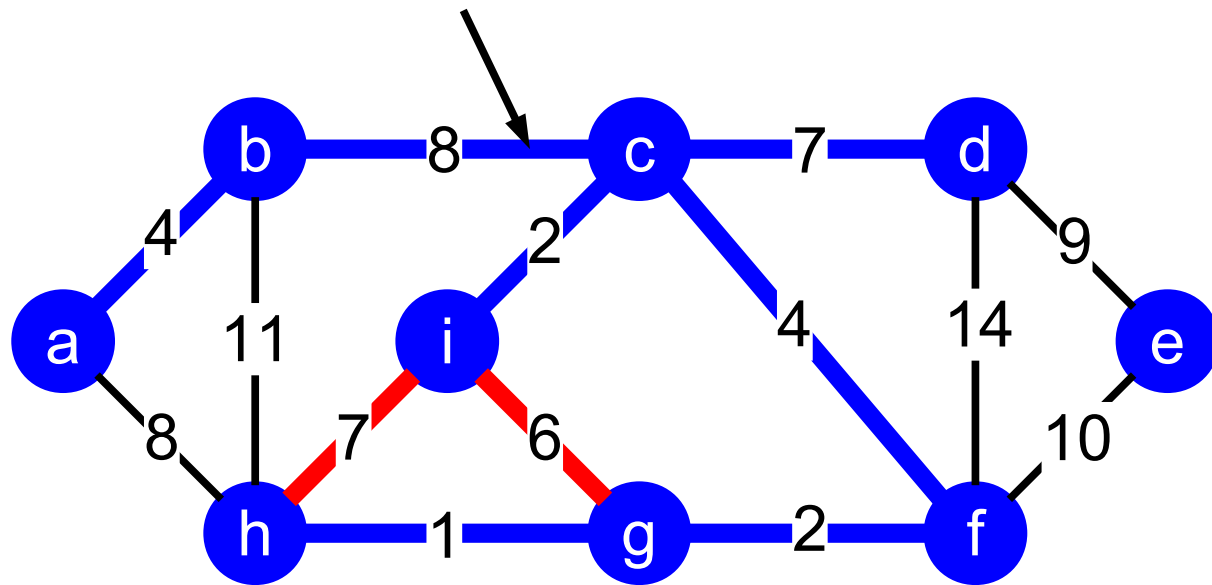
Esempio

Algoritmo di Kruskal



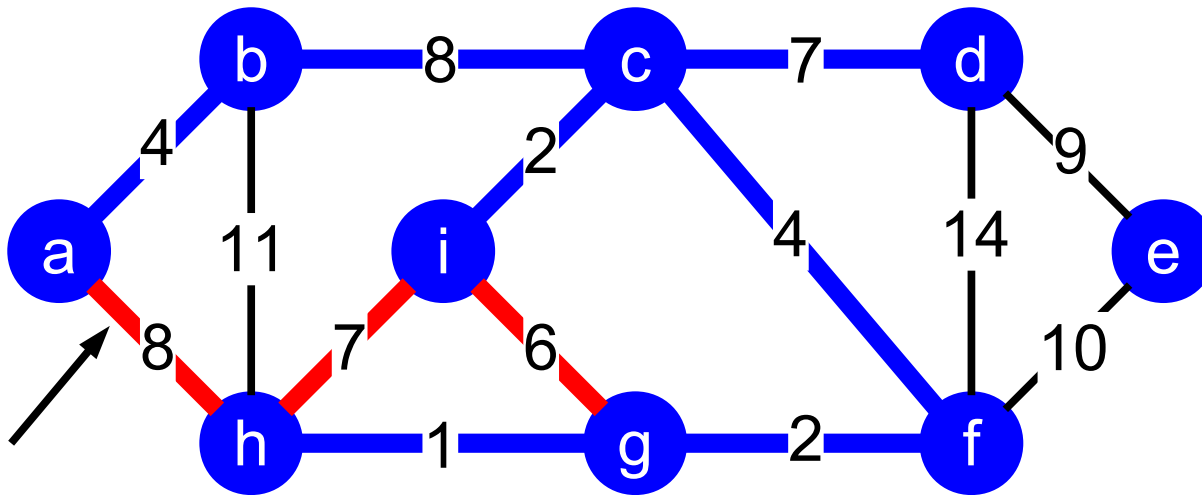
Esempio

Algoritmo di Kruskal



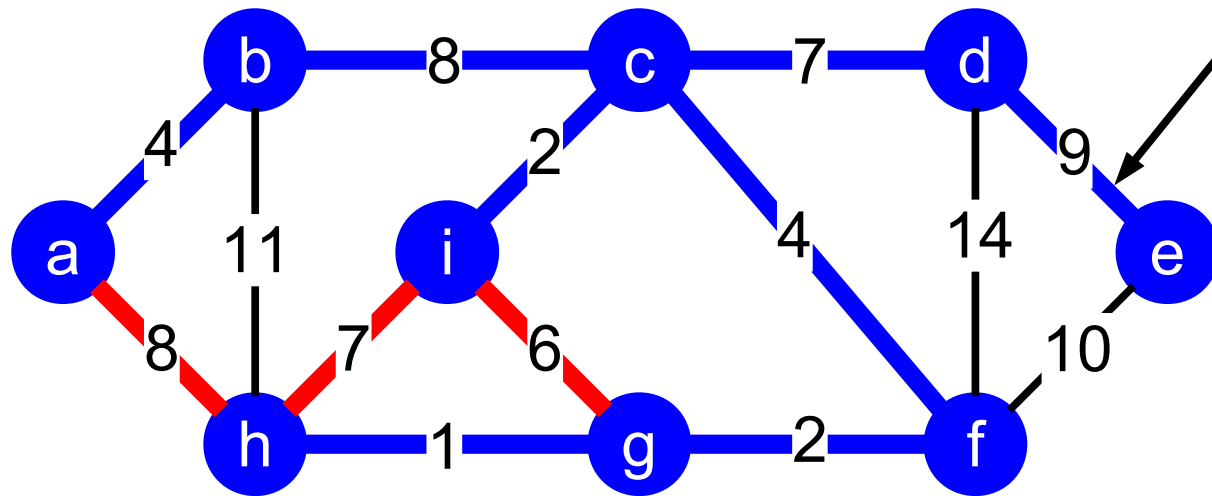
Esempio

Algoritmo di Kruskal



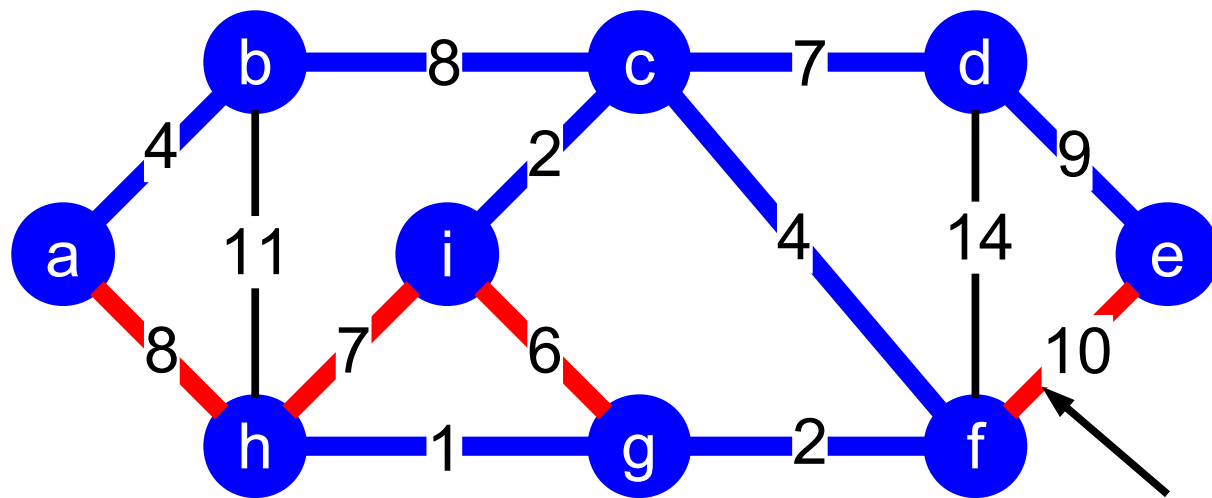
Esempio

Algoritmo di Kruskal



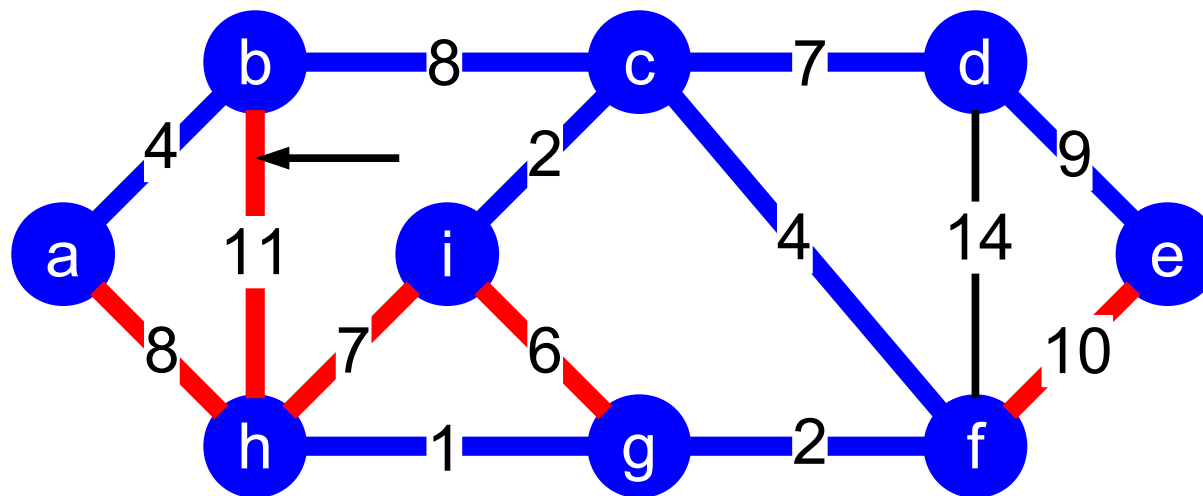
Esempio

Algoritmo di Kruskal



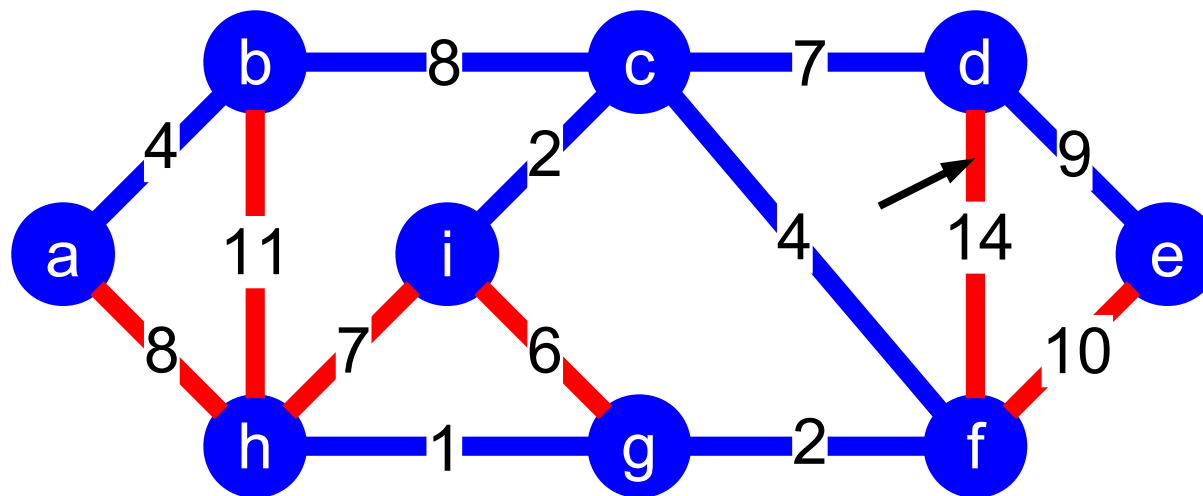
Esempio

Algoritmo di Kruskal



Esempio

Algoritmo di Kruskal



- Finito! Il MST è composto dai soli archi blu

Implementazione

- Ordinare gli archi in ordine non decrescente di peso
 - Sappiamo come fare
- Determinare se gli estremi di un arco appartengono allo stesso albero oppure no
 - Anche qui, sappiamo come fare...
 - ...usando le strutture union-find!

Algoritmo di Kruskal

```
Tree Kruskal-MST(Grafo G=(V,E,w) )  
  UnionFind UF  
  Tree T ← albero vuoto  
  for i ← 1 to G.numNodi() do UF.makeSet(i)  
  // ordina gli archi di E per peso w crescente  
  sort(E, w)  
  for each {u,v} in E do  
    Tu ← UF.find(u)  
    Tv ← UF.find(v)  
    if (Tu ≠ Tv) then           // evita i cicli  
      T ← T ∪ {u, v}           // aggiungi arco  
      UF.union(Tu, Tv)         // unisci componenti  
    endif  
  endfor  
  return T
```

Analisi

- L'ordinamento richiede
$$O(m \log m) = O(m \log n^2) = O(m \log n)$$
dove m è il numero di archi e n il numero di nodi
- Il tempo di esecuzione dipende dalla realizzazione della struttura dati per insiemi disgiunti
 - Vengono effettuate n makeSet, $2m$ find e $(n - 1)$ union
- Se usiamo quickUnion con euristica sul rango, la sequenza di operazioni costa in tutto $O(n+m \log n+n)$
- Totale: $O(2m \log n + 2n) = O(m \log n)$

In un grafo connesso si ha sempre $m \geq n - 1$

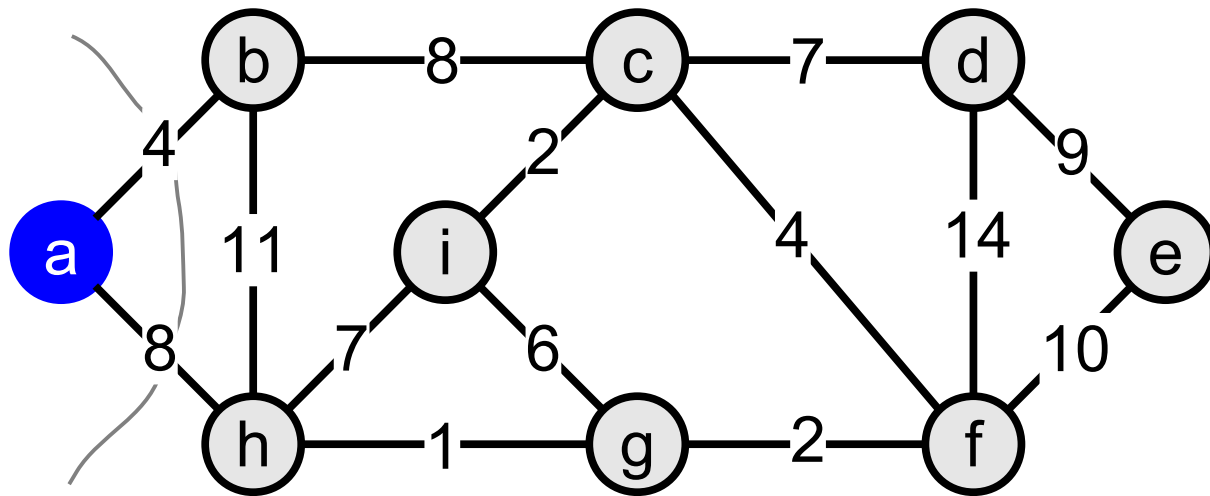
Algoritmo di Prim

- L'algoritmo di Prim utilizza solo la regola del taglio
 - L'ordine di applicazione della regola dipende da un nodo r , detto *radice*, da cui si assume di far partire l'algoritmo
- Si procede mantenendo in un singolo albero T che viene fatto via via “crescere”
 - L'albero parte da un nodo arbitrario r (la *radice*) e cresce fino a quando ricopre tutti i vertici
 - Ad ogni passo viene aggiunto l'arco di peso minimo che collega un nodo già raggiunto dell'albero con uno non ancora raggiunto

R. C. Prim: *Shortest connection networks and some generalizations*.
In: Bell System Technical Journal, 36 (1957), pp. 1389–1401

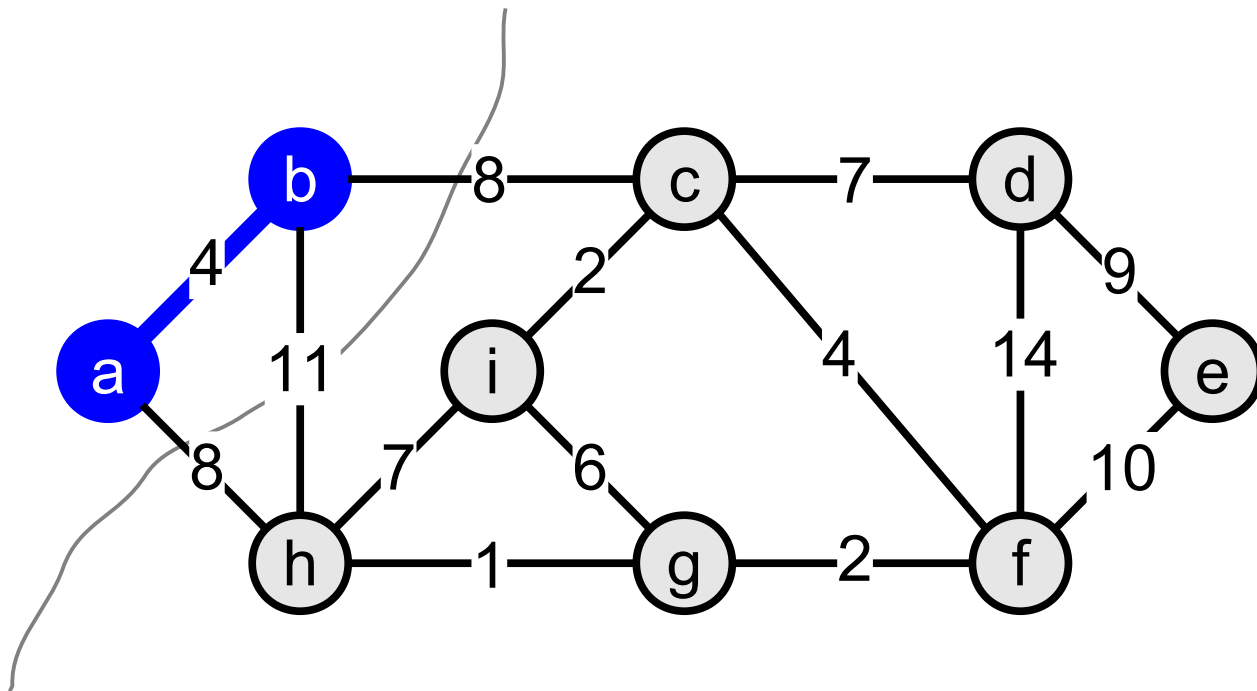
Esempio

Algoritmo di Prim



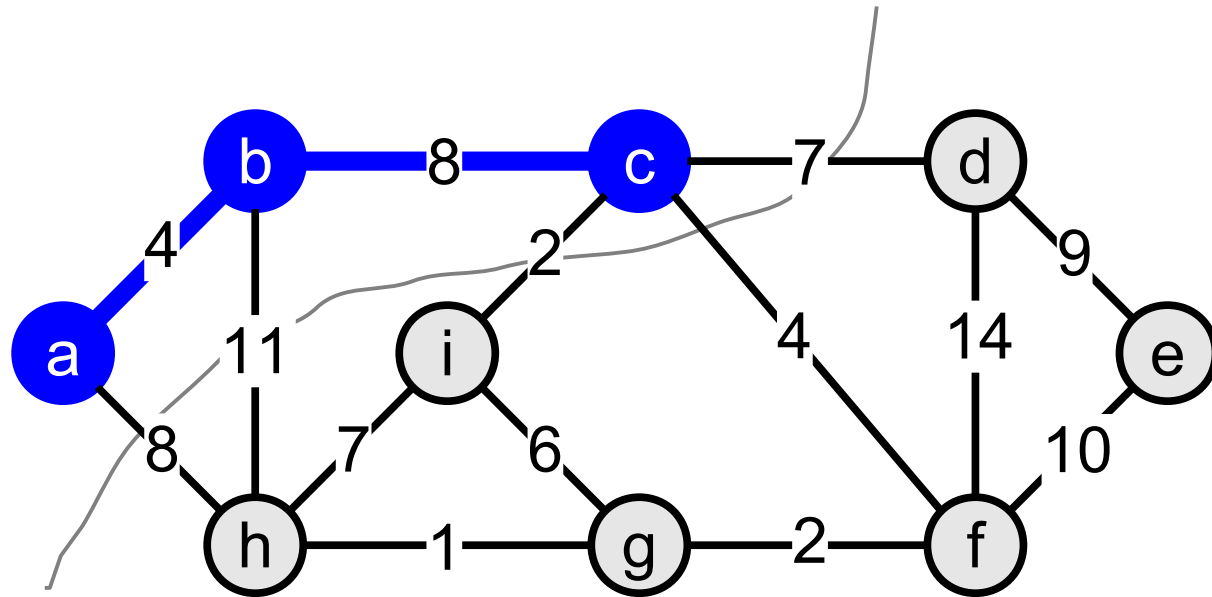
Esempio

Algoritmo di Prim



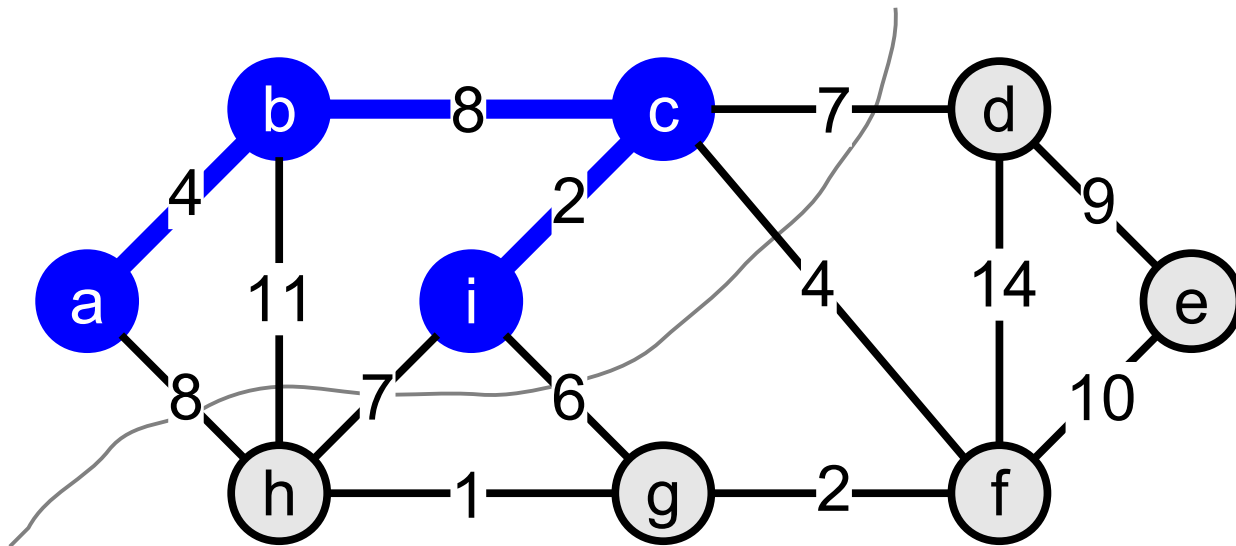
Esempio

Algoritmo di Prim



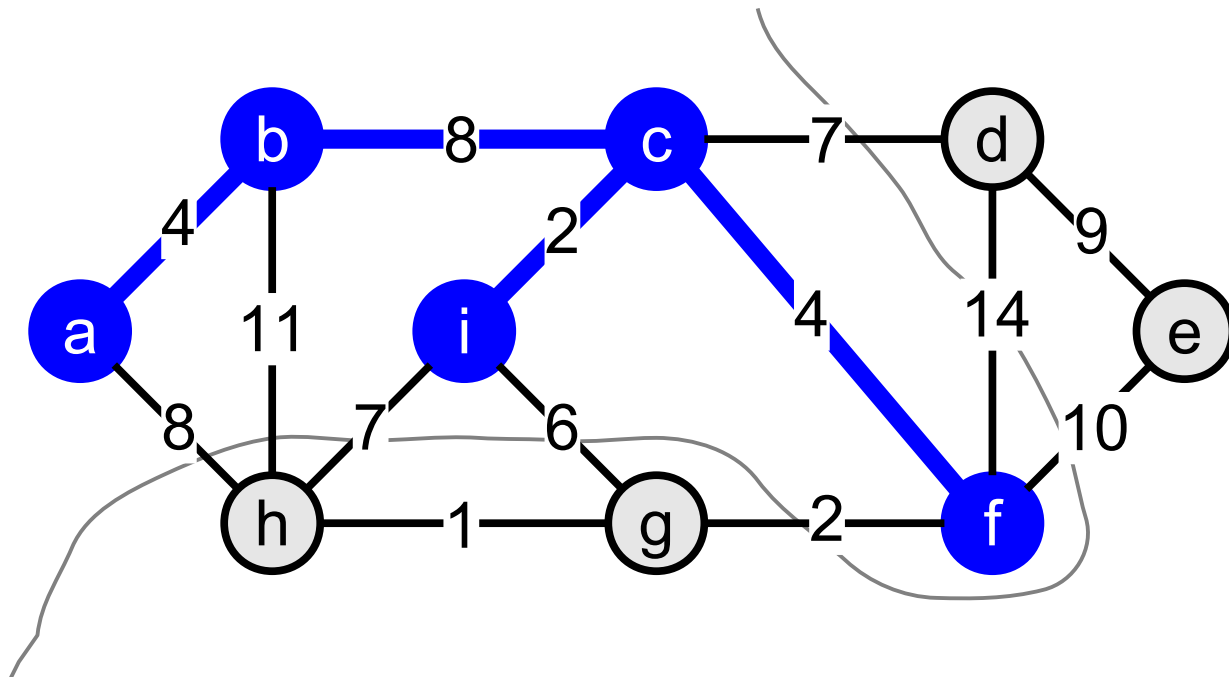
Esempio

Algoritmo di Prim



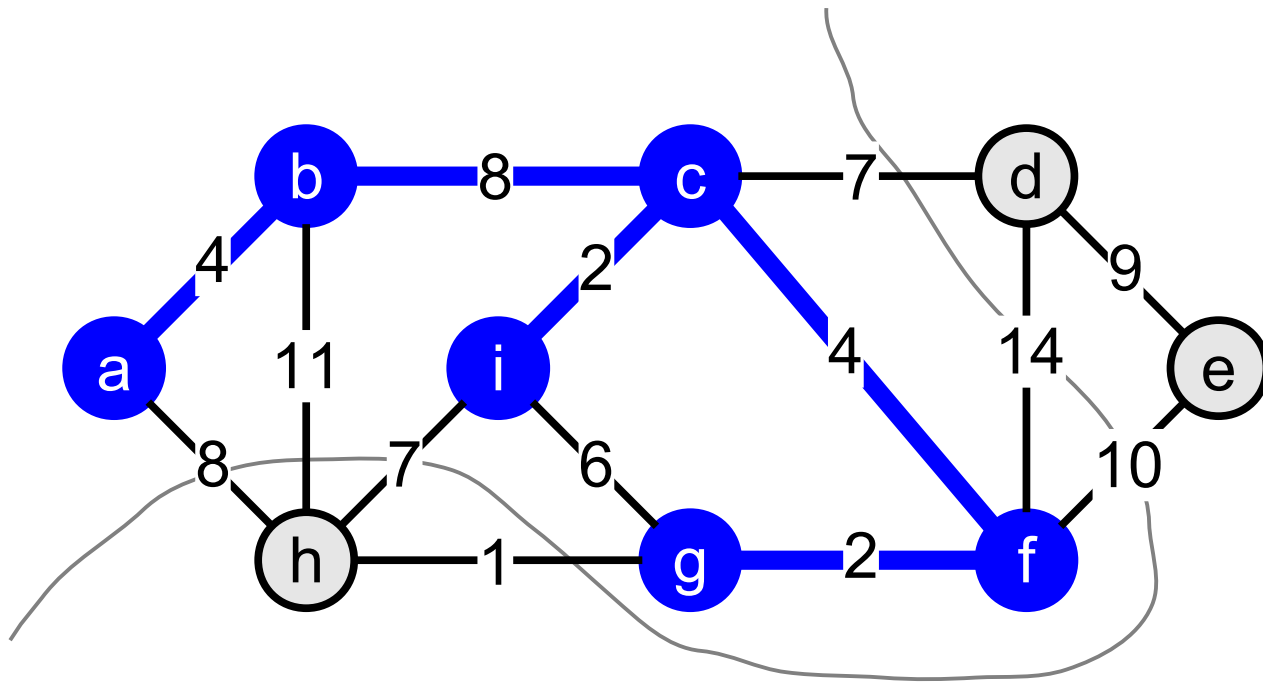
Esempio

Algoritmo di Prim



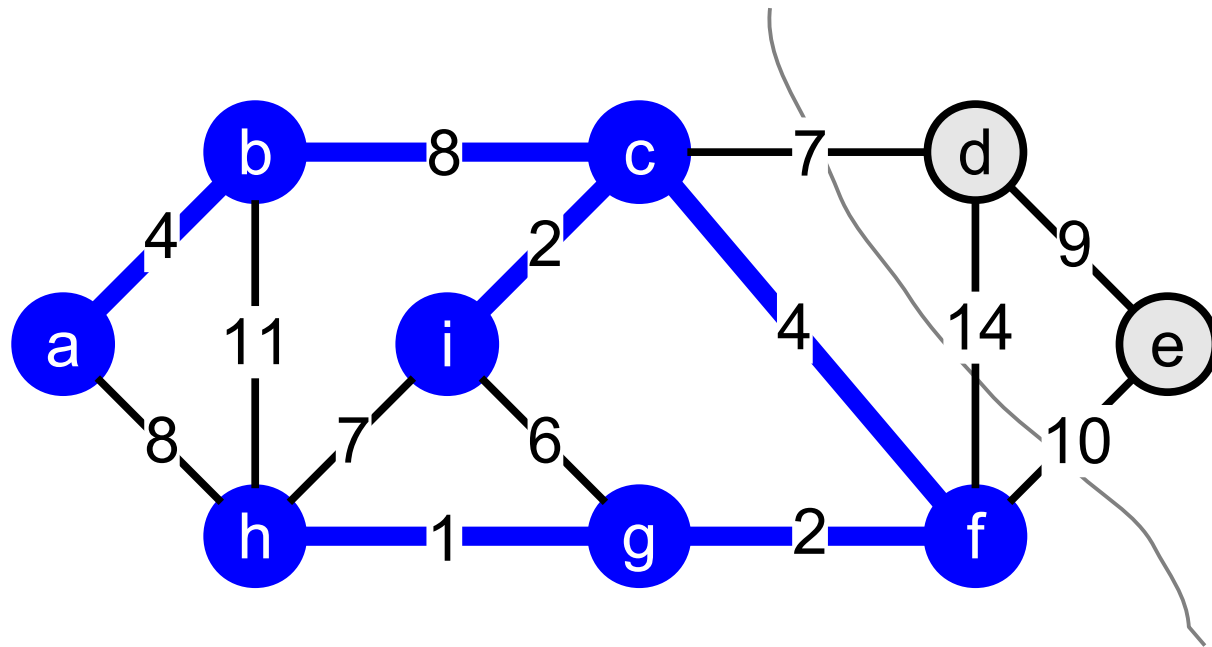
Esempio

Algoritmo di Prim



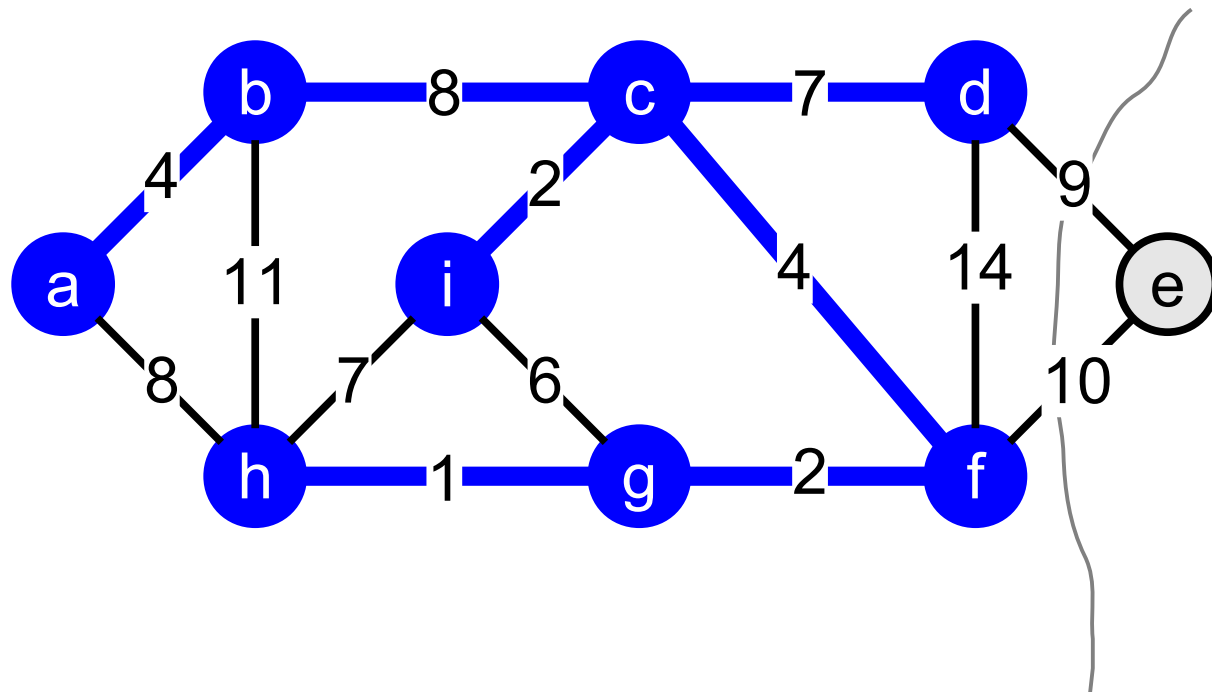
Esempio

Algoritmo di Prim



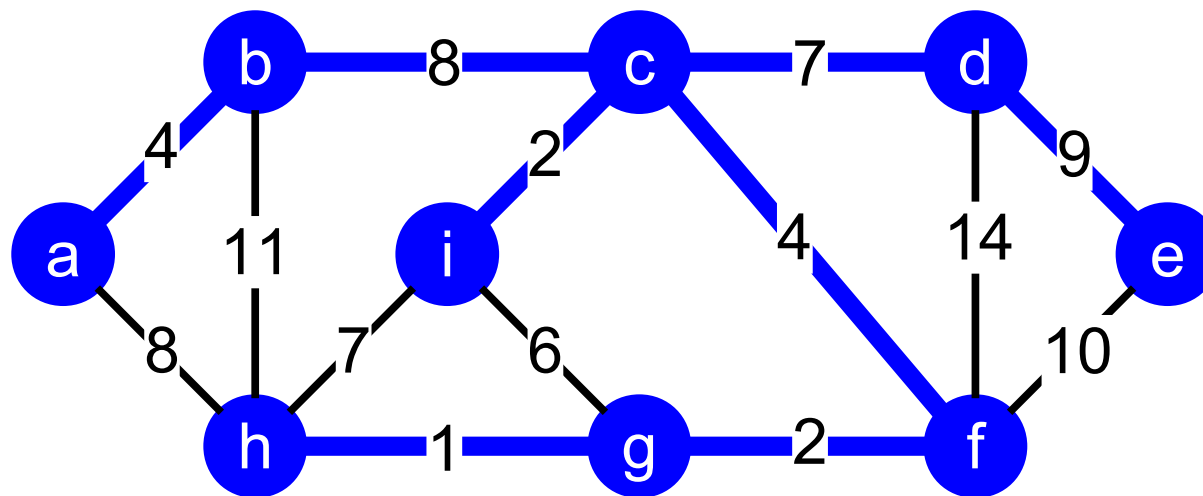
Esempio

Algoritmo di Prim



Esempio

Algoritmo di Prim



Implementazione

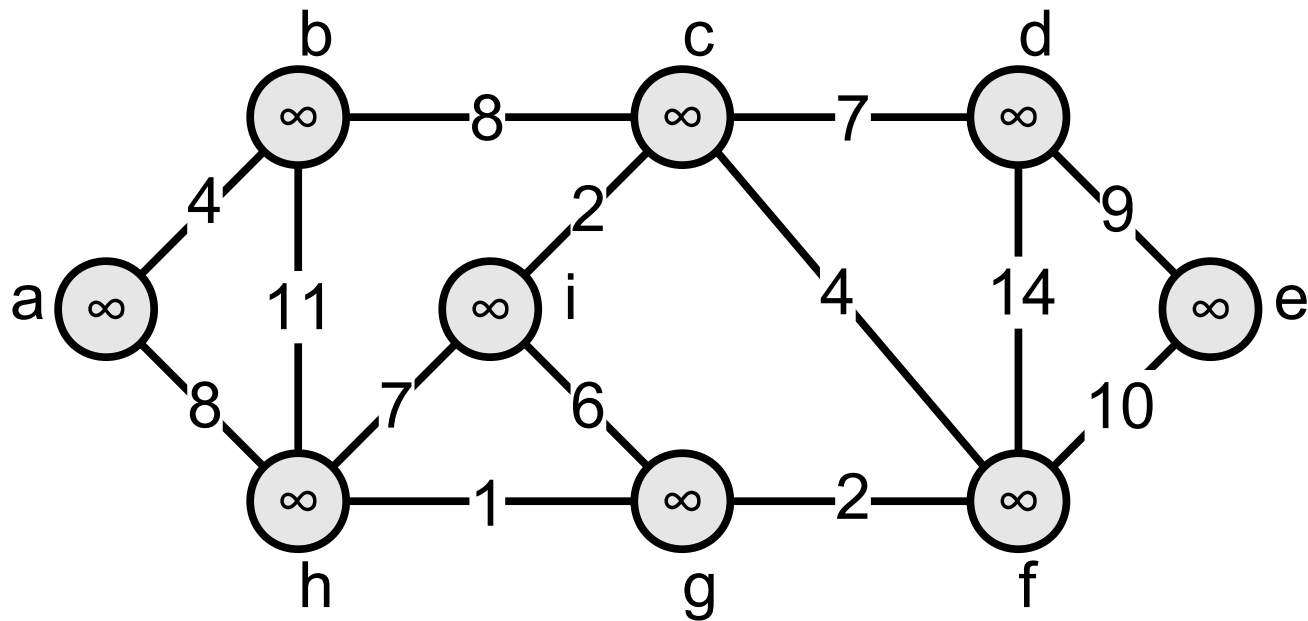
- Una struttura dati per i nodi non ancora nell'albero
 - i nodi già osservati ma non ancora nel MST si trovano in una *coda con priorità Q* ordinata in base ad un valore *$v.dist$*
 - Più precisamente, la coda viene pian piano popolata quando un nodo risulta essere collegato ad un nodo già nel MST
 - *$v.dist$* è il peso minimo di un arco che collega il nodo *v* , che non appartiene all'albero, ad un nodo già nell'albero
 - $+\infty$ se tale arco non esiste (in questo caso il nodo non è ancora entrato nella coda con priorità)
- Albero rappresentato con il puntatore al padre *$v.parent$*
- Per ogni vertice un booleano *$v.bool$* dice se è già nel MST
- Terminazione: quando la coda Q è vuota
 - Tutti i nodi tranne la radice conoscono il proprio padre

Algoritmo di Prim

```
integer[] Prim-MST(Grafo G=(V,E,w), nodo s)
  for each v ∈ V do
    v.dist ← ∞;
    v.parent ← null;
    v.bool ← false;
  endfor
  s.dist ← 0;
  CodaPriorita<integer, double> Q; Q.insert(s, s.dist);
  while (not Q.isEmpty()) do
    u ← Q.find(); Q.deleteMin(); u.bool ← true;
    for each v ∈ u.adj do
      if (not v.bool) then
        if (v.dist == ∞) then
          Q.insert(v, w(u,v));
          v.dist ← w(u,v);
          v.parent ← u;
        elseif (w(u,v) < v.dist) then
          Q.decreaseKey(v, w(u,v));
          v.dist ← w(u,v);
          v.parent ← u;
        endif
      endif
    endfor
  endwhile
```

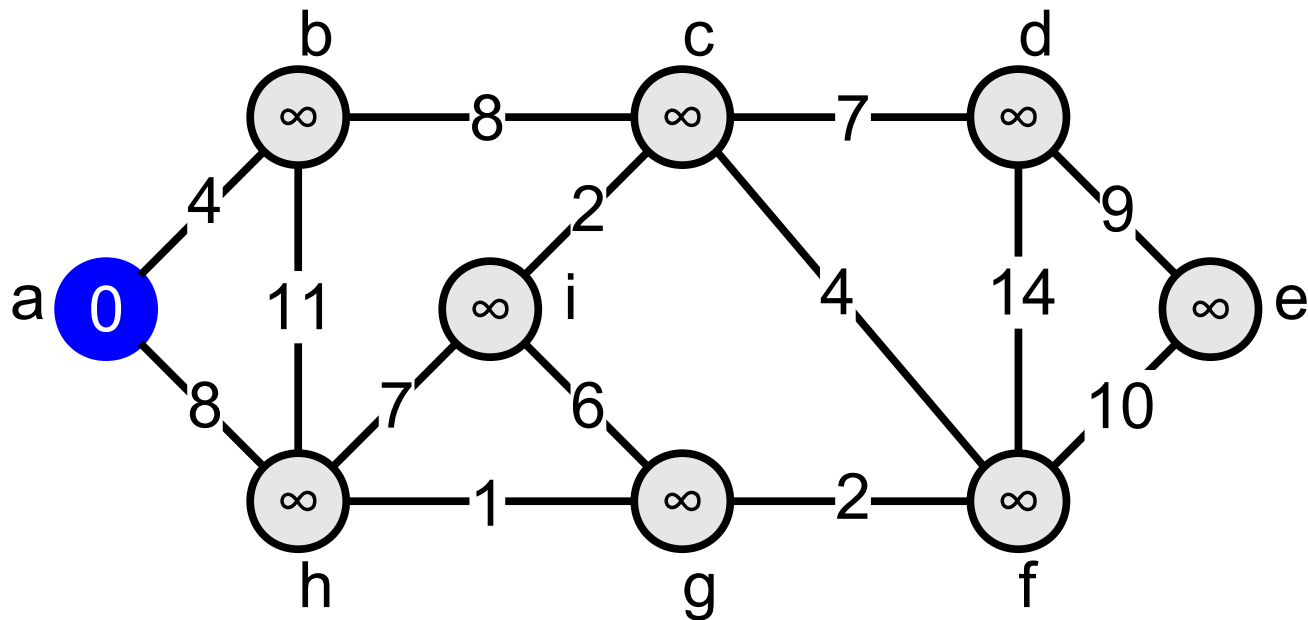
Esempio di esecuzione

$Q = \{\}$



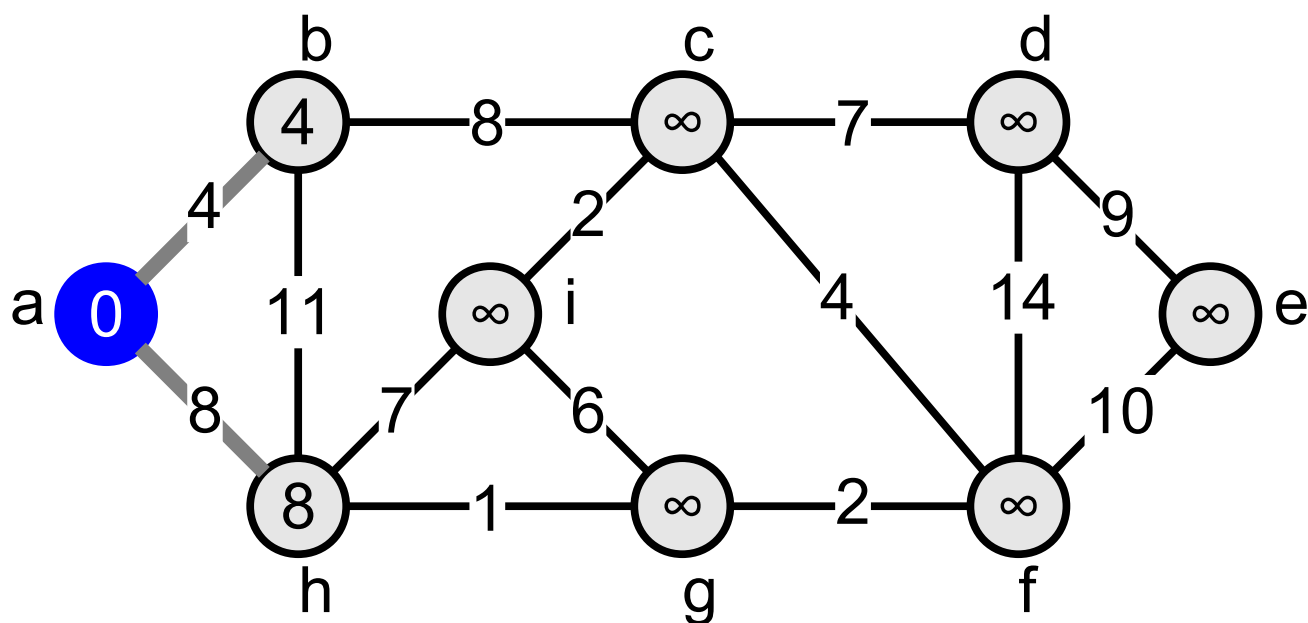
Esempio di esecuzione

$$Q = \{ (a, 0) \}$$



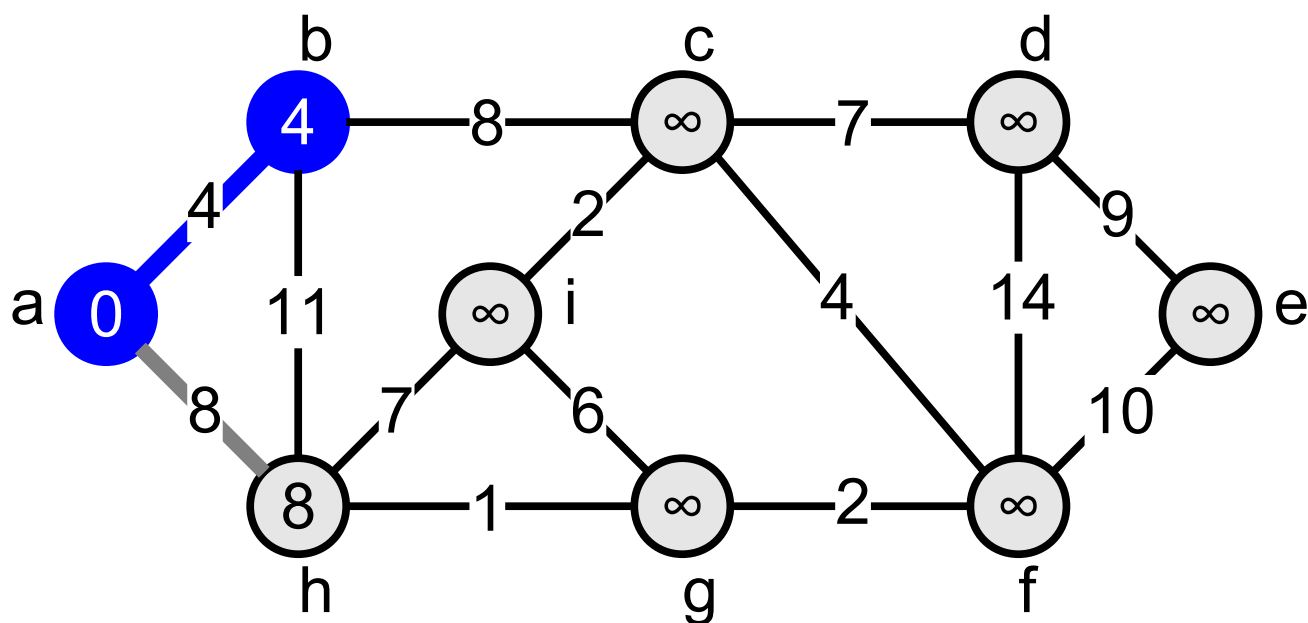
Esempio di esecuzione

$$Q = \{ (b,4), (h,8) \}$$



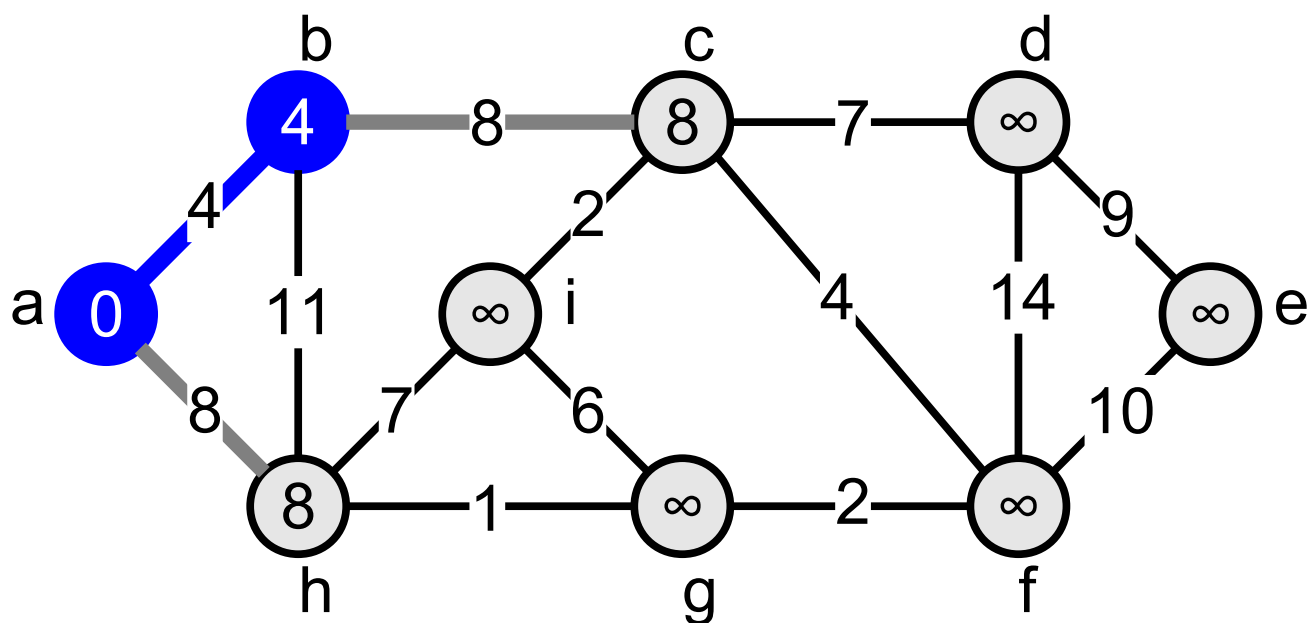
Esempio di esecuzione

$$Q = \{ (b,4), (h,8) \}$$



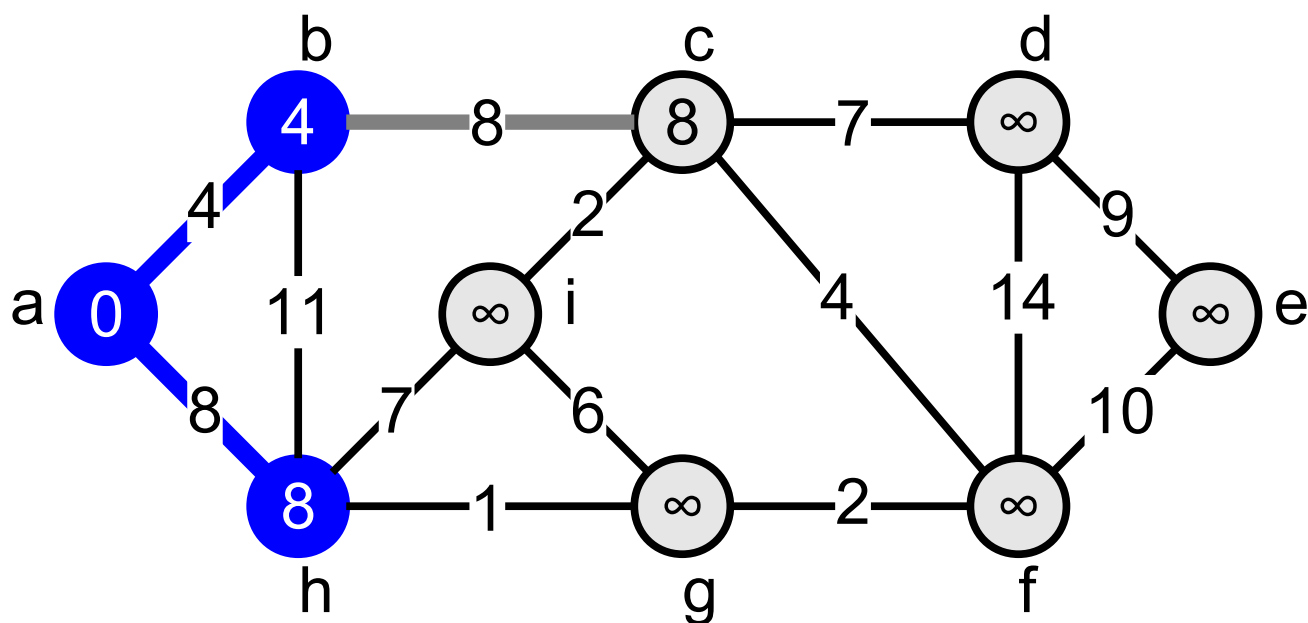
Esempio di esecuzione

$$Q = \{ (h,8), (c,8) \}$$



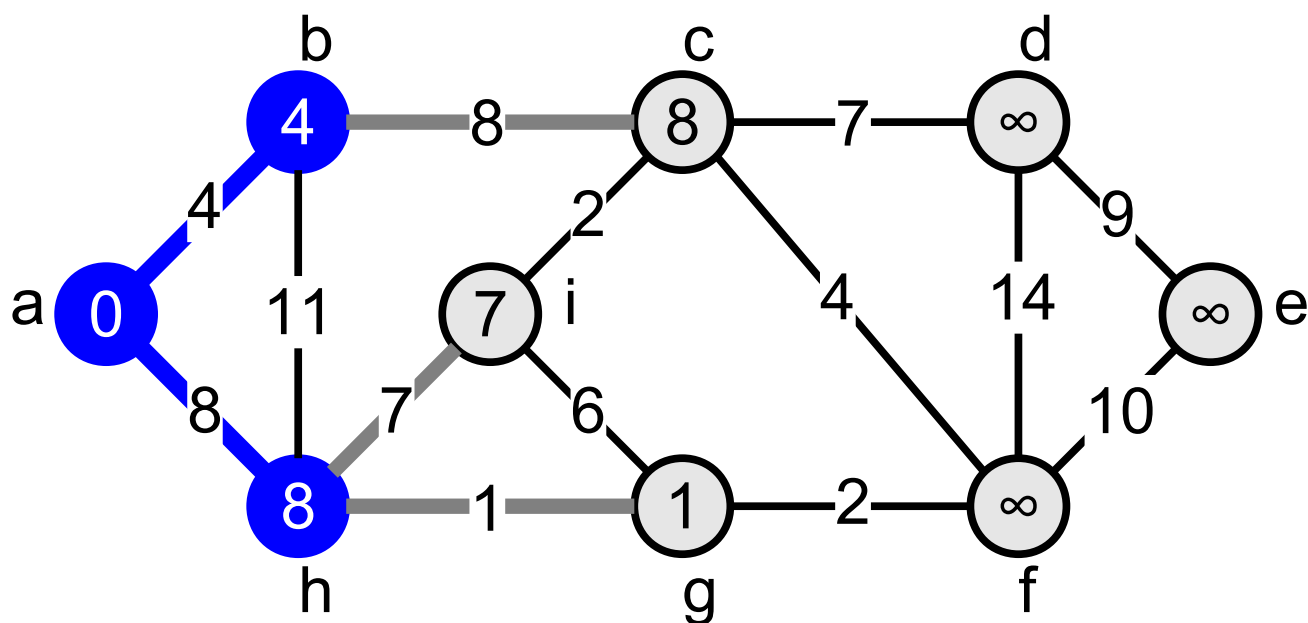
Esempio di esecuzione

$$Q = \{ (h, 8), (c, 8) \}$$



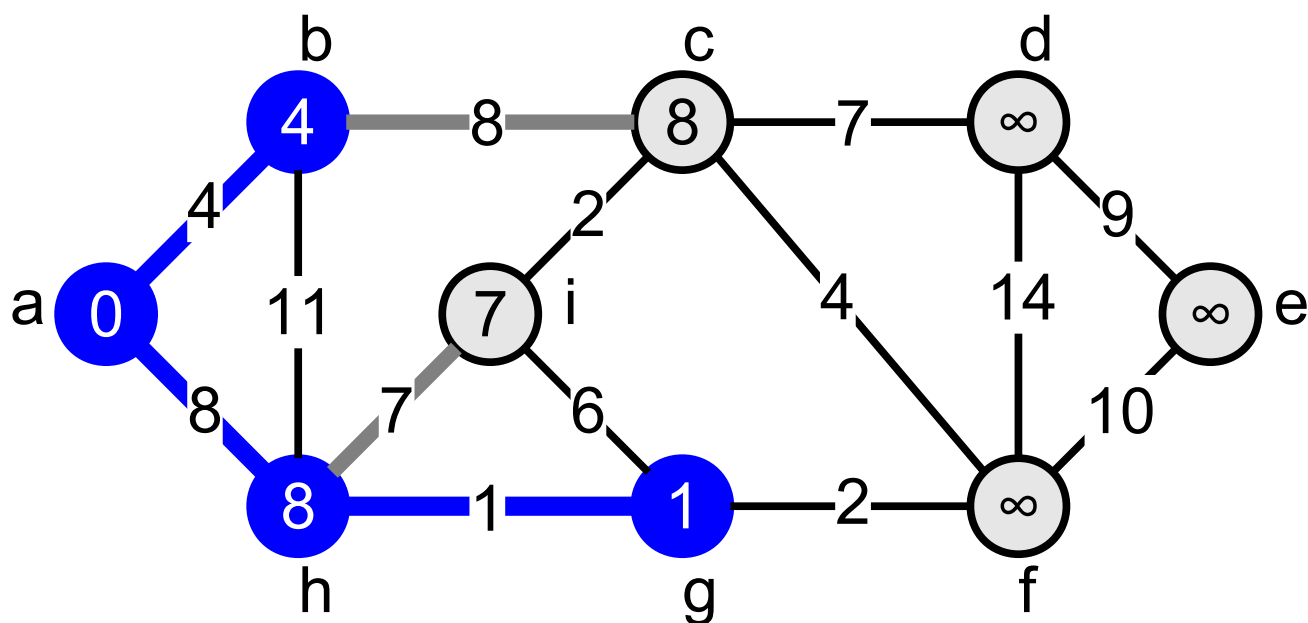
Esempio di esecuzione

$$Q = \{ (g,1), (i,7), (c,8) \}$$



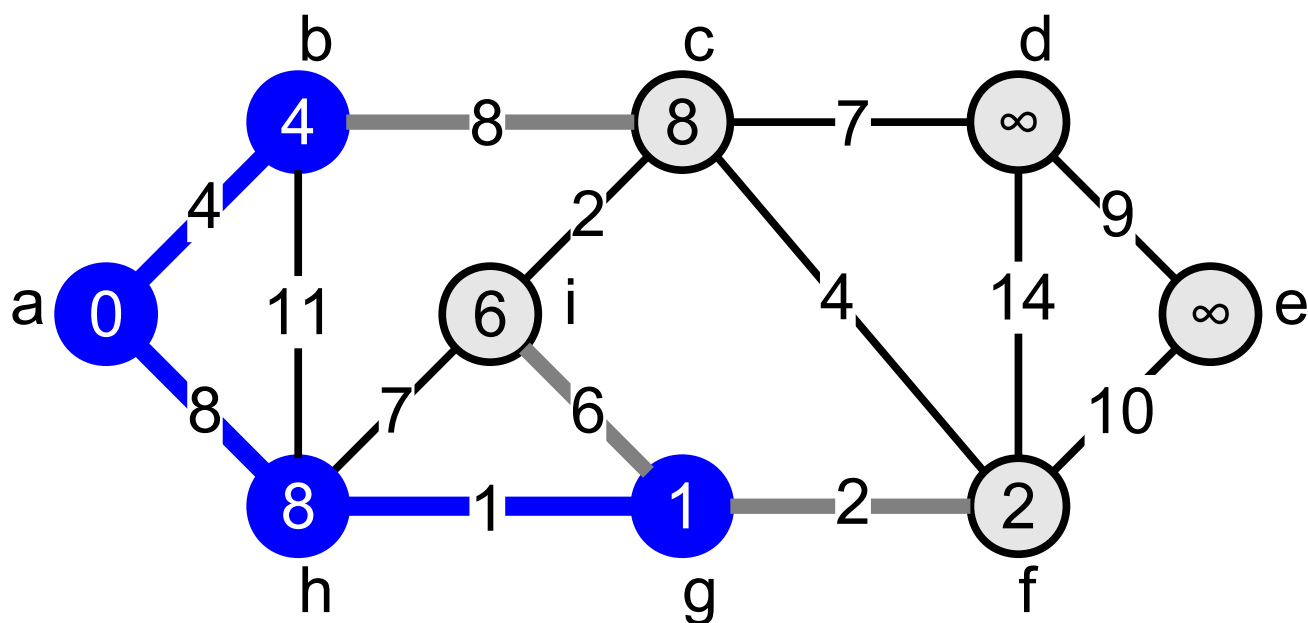
Esempio di esecuzione

$$Q = \{ (g, 1), (i, 7), (c, 8) \}$$



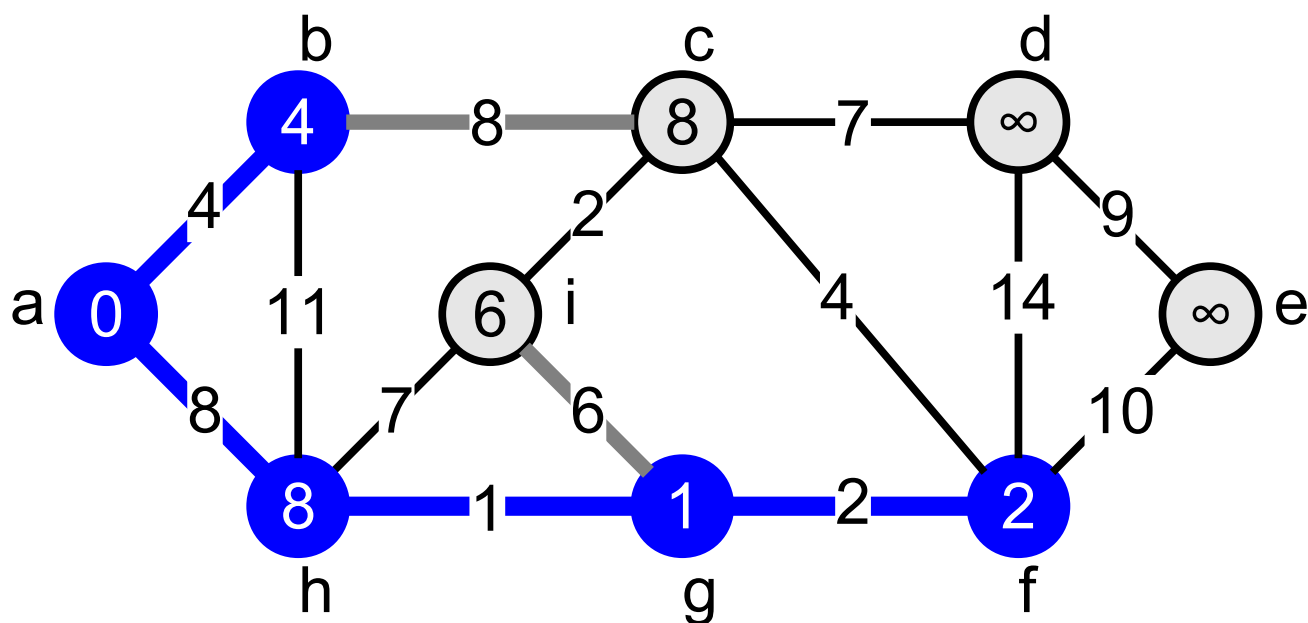
Esempio di esecuzione

$$Q = \{ (f,2), (i,6), (c,8) \}$$



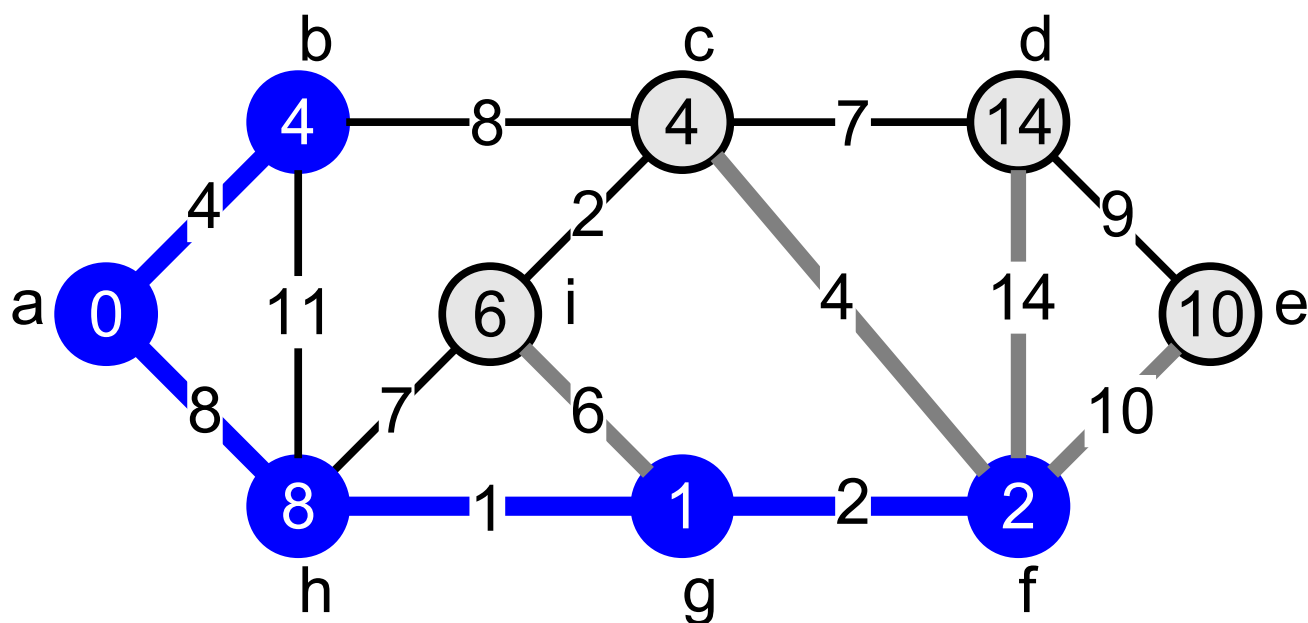
Esempio di esecuzione

$$Q = \{ (f,2), (i,6), (c,8) \}$$



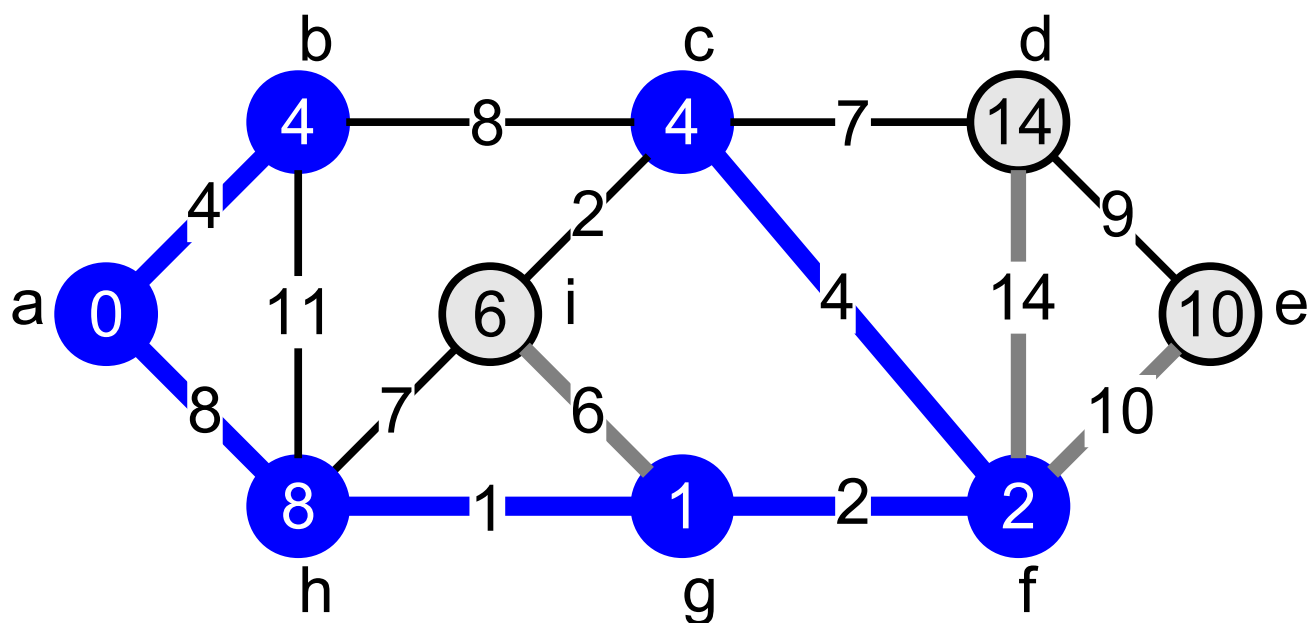
Esempio di esecuzione

$$Q = \{ (c,4), (i,6), (e,10), (d,14) \}$$



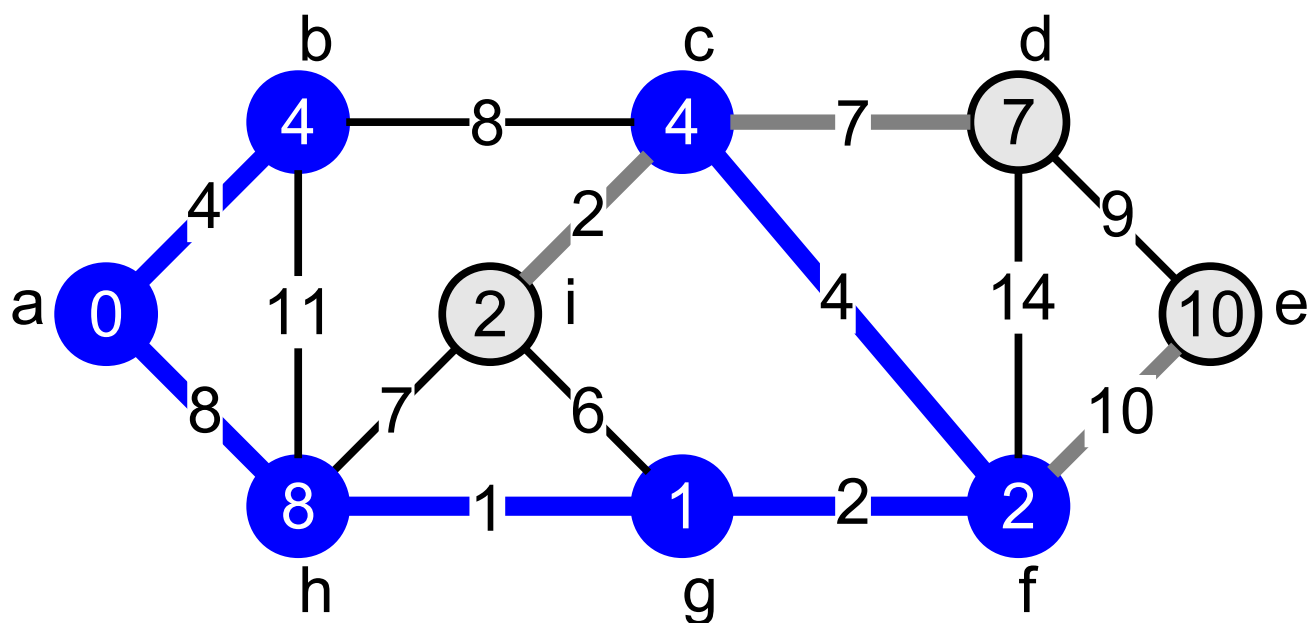
Esempio di esecuzione

$Q = \{ (c,4), (i,6), (e,10), (d,14) \}$



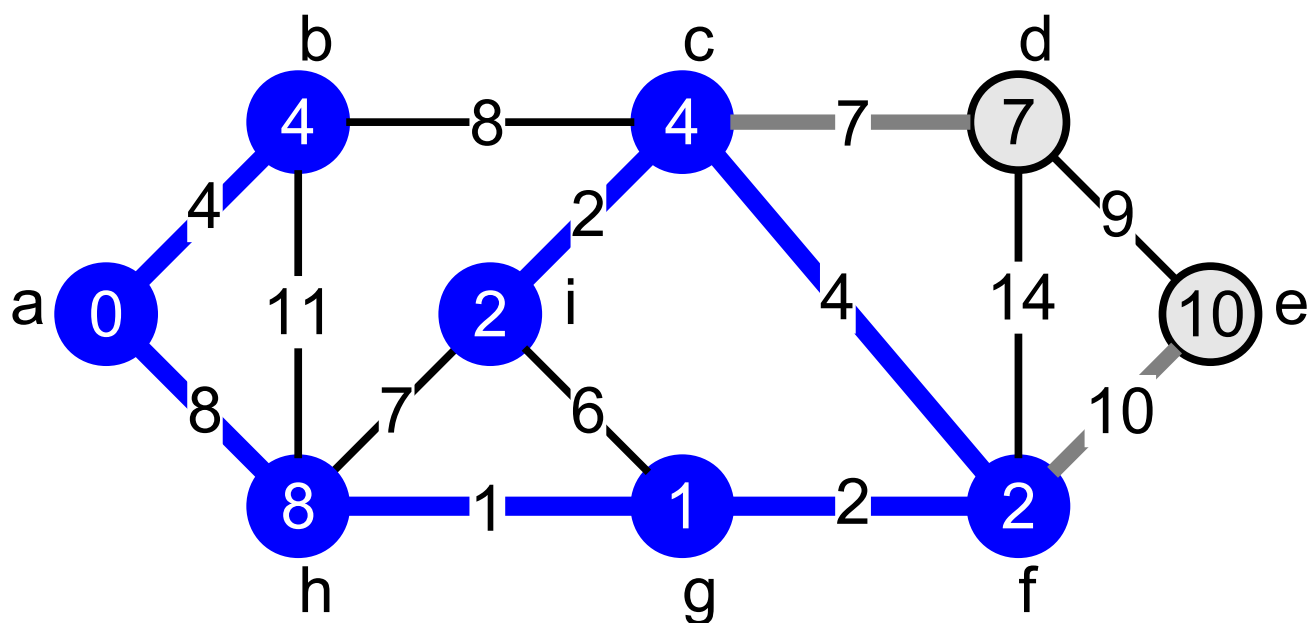
Esempio di esecuzione

$$Q = \{ (i,2), (d,7), (e,10) \}$$



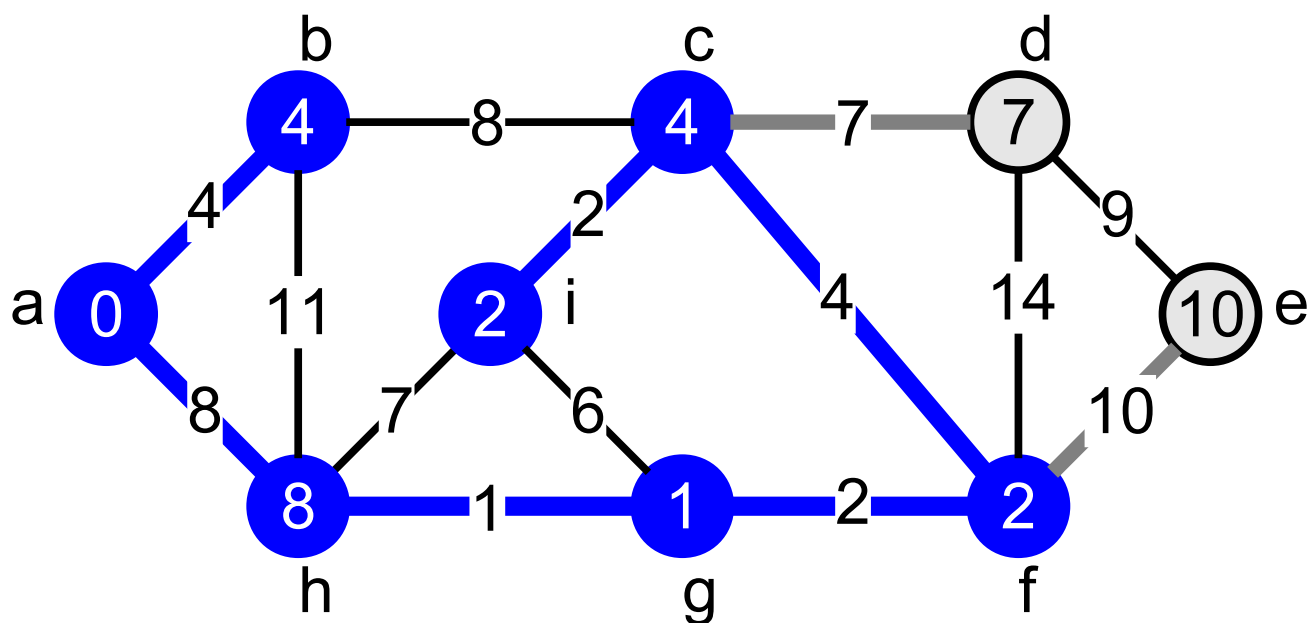
Esempio di esecuzione

$$Q = \{ (i, 2), (d, 7), (e, 10) \}$$



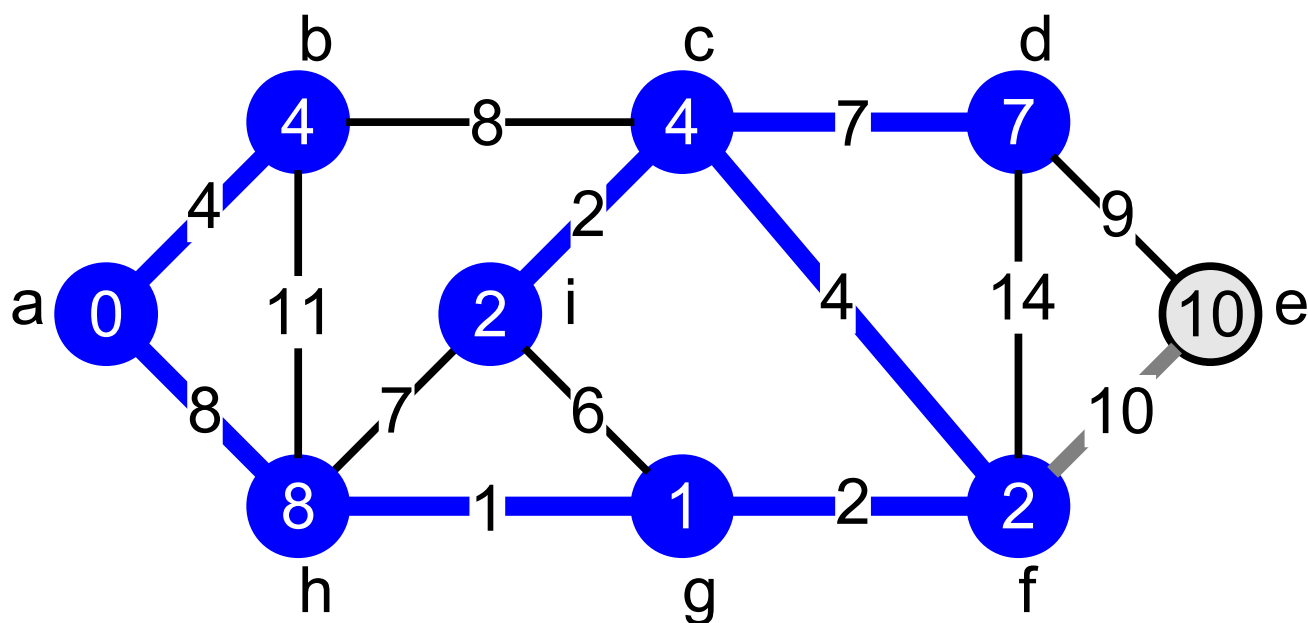
Esempio di esecuzione

$$Q = \{ (d,7), (e,10) \}$$



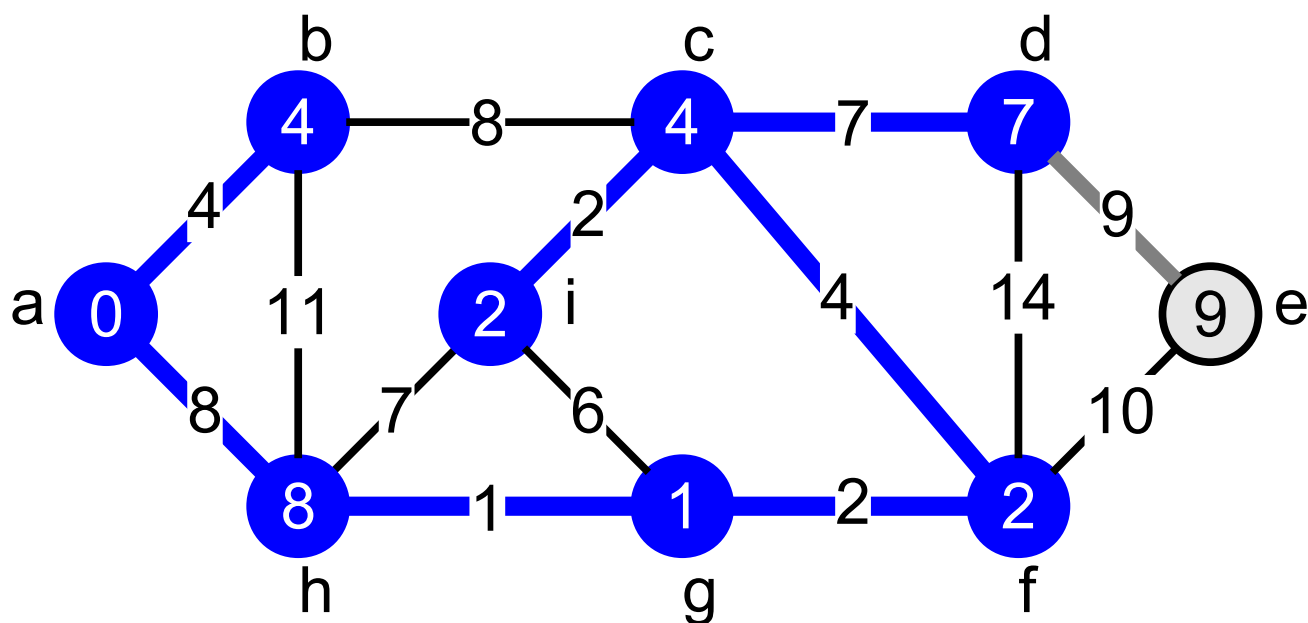
Esempio di esecuzione

$$Q = \{ (d, 7), (e, 10) \}$$



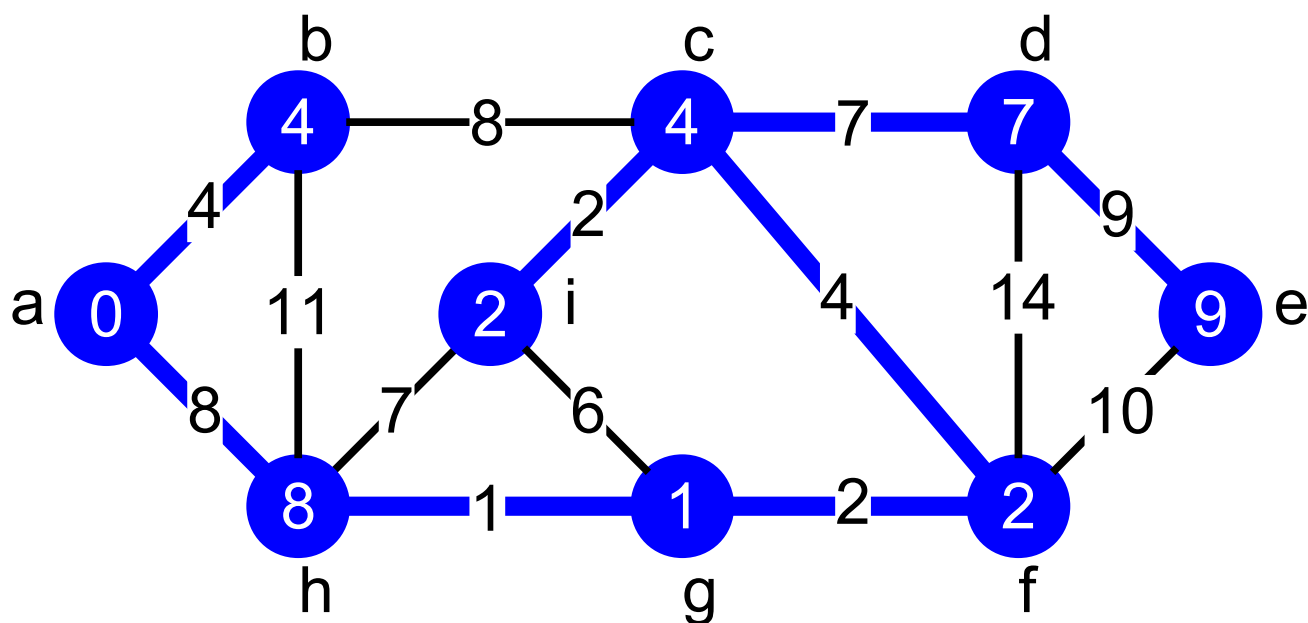
Esempio di esecuzione

$$Q = \{ (e, 9) \}$$



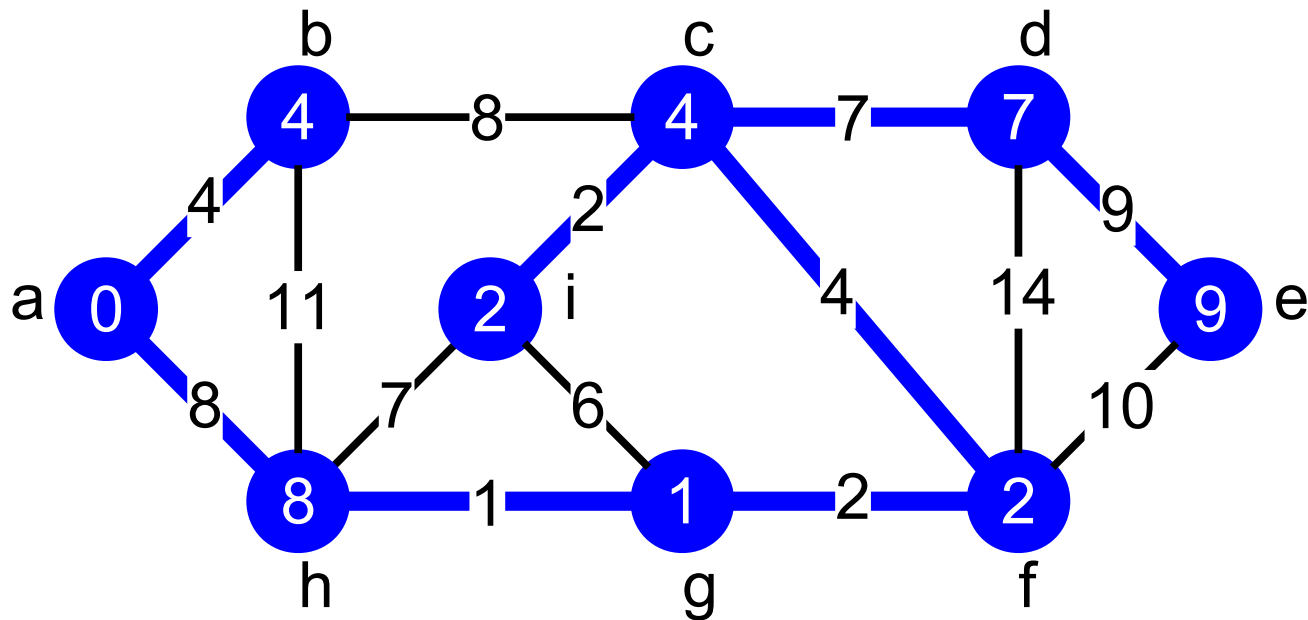
Esempio di esecuzione

$$Q = \{ (e, 9) \}$$



Esempio di esecuzione

$Q = \{\}$



Algoritmo di Prim

```
integer[] Prim-MST(Grafo G=(V,E,w), nodo s)
  for each v ∈ V do
    v.dist ← ∞;
    v.parent ← null;
    v.bool ← false;
  endfor
  s.dist ← 0;
  CodaPriorita<integer, double> Q; Q.insert(s, s.dist);
  while (not Q.isEmpty()) do
    u ← Q.find(); Q.deleteMin(); u.bool ← true;
    for each v ∈ u.adj do
      if (not v.bool) then
        if (v.dist == ∞) then
          Q.insert(v, w(u,v));
          v.dist ← w(u,v);
          v.parent ← u;
        elseif (w(u,v) < v.dist) then
          Q.decreaseKey(v, w(u,v));
          v.dist ← w(u,v);
          v.parent ← u;
        endif
      endif
    endfor
  endwhile
```

Algoritmo di Prim

```
integer[] Prim-MST(Grafo G=(V,E,w), nodo s)
  for each v ∈ V do
    v.dist ← ∞;
    v.parent ← null;
    v.bool ← false;
  endfor
  s.dist ← 0;
  CodaPriorita<integer, double> Q; Q.insert(s, s.dist);
  while (not Q.isEmpty()) do
    u ← Q.find(); Q.deleteMin(); u.bool ← true;
    for each v ∈ u.adj do
      if (not v.bool) then
        if (v.dist == ∞) then
          Q.insert(v, w(u,v));
          v.dist ← w(u,v);
          v.parent ← u;
        elseif (w(u,v) < v.dist) then
          Q.decreaseKey(v, w(u,v));
          v.dist ← w(u,v);
          v.parent ← u;
        endif
      endif
    endfor
  endwhile
```

n deleteMin()

*n insert()
(inclusa Q.insert(s,0))*

O(m) decreaseKey()

Algoritmo di Prim

Costo computazionale

- Utilizzando una coda di priorità basata su min-heap
 - n deleteMin() costano $O(n \log n)$
 - n insert() costano $O(n \log n)$
 - $O(m)$ decreaseKey() costano $O(m \log n)$
- Totale
 - $O(n \log n + n \log n + m \log n) =$
 $O(m \log n + n \log n) =$
 $O(m \log n)$

In un grafo connesso si ha
sempre $m \geq n - 1$